

Automatic Image Analysis

Generative Adversarial Networks

Ref.: Prince: Understanding Deep Learning

Unsupervised Learning

- Supervised learning:
models map observed data $\mathbf{x}$ to output labels $\mathbf{y}$
- Unsupervised learning:
Learning in the absence of labels.
- Which of the preceding chapters
was concerned with one or the other?

Terminology

- **Supervised Learning:** training data comes with labels
 - Standard method: classification
- **Unsupervised Learning:** data without labels
 - Standard method: clustering
- **Self-Supervised Learning**
 - Labels are generated without human intervention
 - from the data
 - making use of properties of data or data acquisition

Configurations allowing self-supervised learning

- No training data required
- For example
 - Predicting future video frames
 - A future video frame is acquired after some (short) time.
 - Observing the effect of segmentation on low-level image features
 - Assumption: If segmentation causes unusual features, it must be wrong (Jenny & Favaro, 2018; Shimoda & Yanai, 2019).
 - Stereo partnering image as training data
 - The second image is to be predicted as well as acquired by the second camera.
 - Video from motion blur
 - High frame rate camera acquires training data (Jin, Hu, Favaro, 2019).

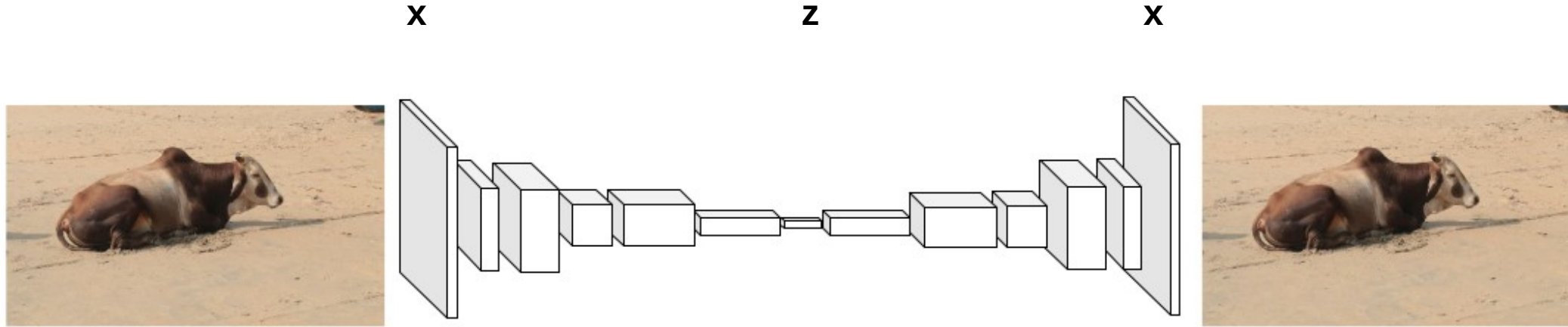
Related Categories and Terminology

- Pre-text task learning
 - as preparation of solving a down-stream task
- Continual learning
- Domain adaptation
- Weakly-supervised learning
 - Incomplete labels
 - Inexact (coarse-grained) labels
 - Inaccurate (not ground truth) noisy labels

Unsupervised Learning

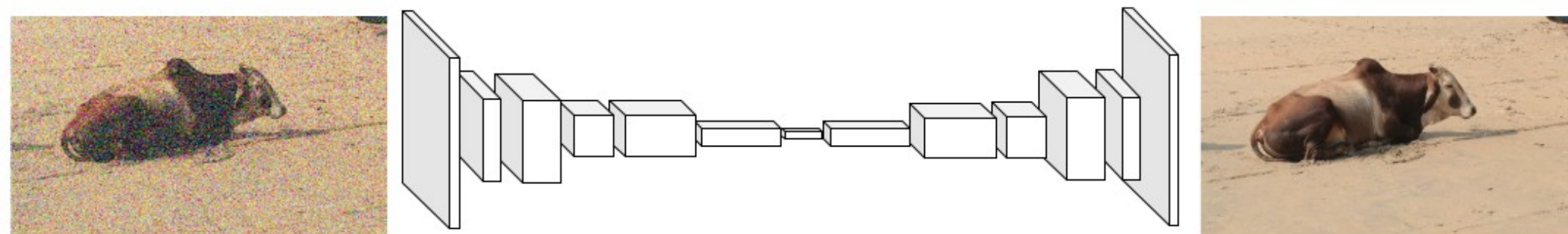
- Common strategy:
mapping between data $\mathbf{x}$ and unseen latent variables
- $\mathbf{z}$ can be seen as a compressed version of $\mathbf{x}$

Pretext Task: Compression



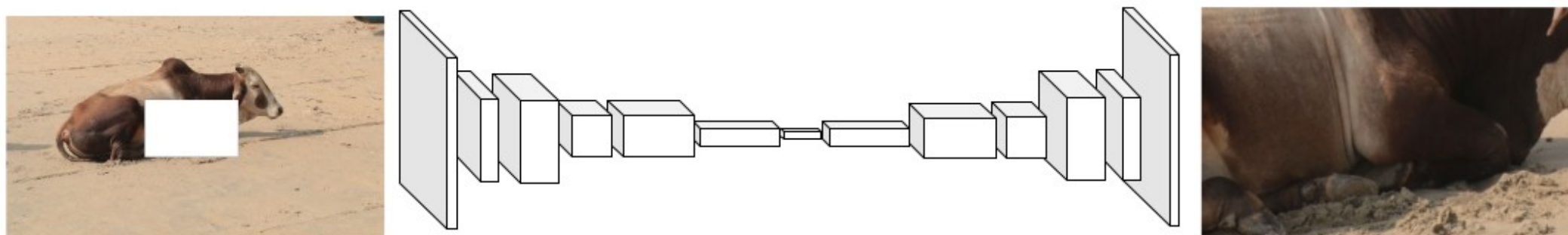
- Train an autoencoding network reconstruct an image after coarse feature layer.
- Use encoding network for downstream task.

Pretext Task: Compression + Reconstruction



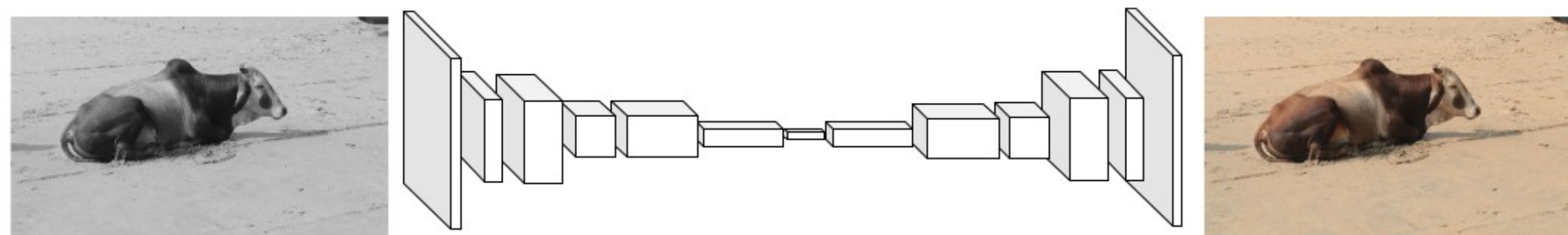
- Same as before but apply distortion function $d(I)$ before feeding the image into the network.
- Use encoding network for downstream task.

Pretext Task: Inpainting



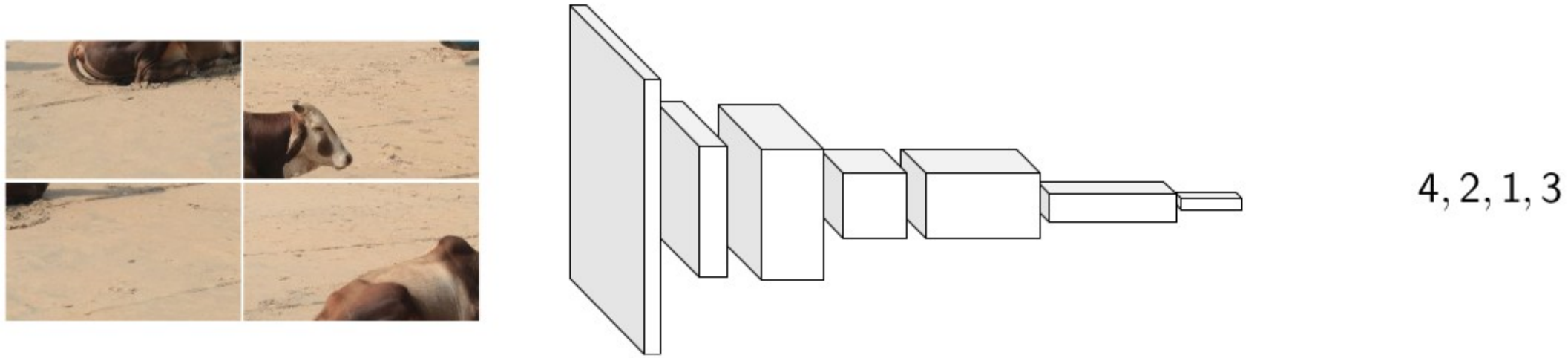
- Predict one part of the data from another.
- Can also be a random part of the image or e.g. the bottom half or frames of a video sequence.
- Context Encoders: Feature Learning by Inpainting, Pathak et al., CVPR 2016

Pretext Task: Colorization



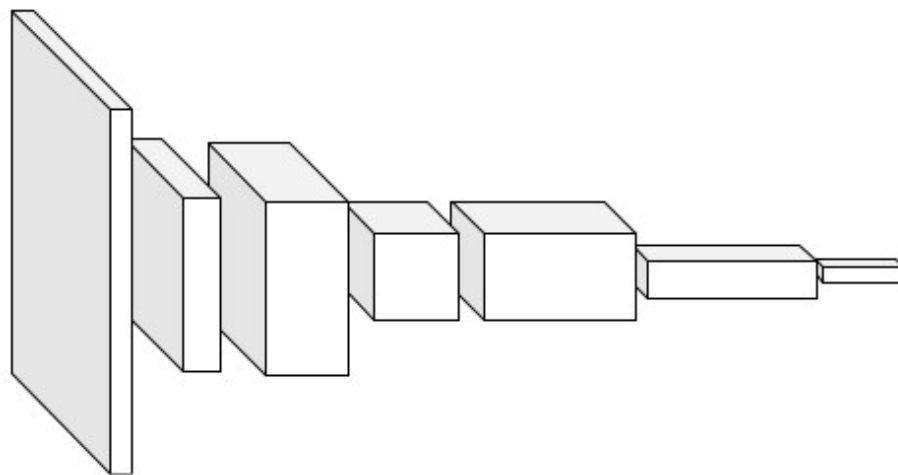
- Similar to inpainting we predict a left-out property the data.
- Colorful Image Colorization, Zhang et al., ECCV 2016
- Tracking Emerges by Colorizing Videos, Vondrick et al., ECCV 2018

Pretext Task: Frame Permutation



- We can also formulate the pretext task as classification problem. Here one of $n!$ possible permutations.
- Can also be done with video frames.
- Unsupervised Learning of Visual Representations by Solving Jigsaw Puzzles, Noroozi & Favaro, ECCV 2016

Pretext Task: Frame Relation



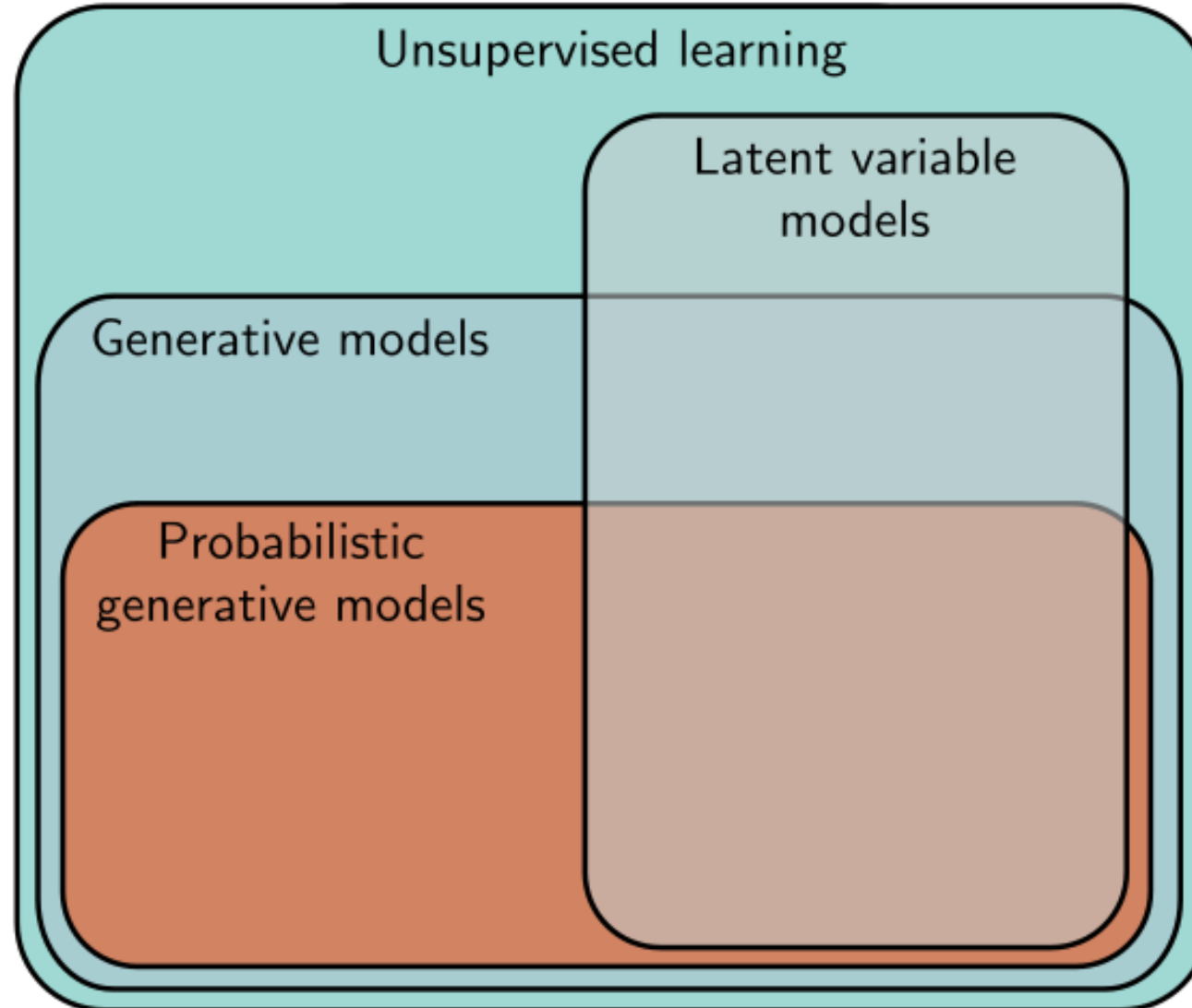
top – right

- Or as a discrete spatial relation
- Unsupervised Visual Representation Learning by Context Prediction, Doersch et al., ICCV 2015

Unsupervised Learning

- Common strategy:
mapping between data $\mathbf{x}$ and unseen latent variables $\mathbf{z}$
- $\mathbf{z}$ can be seen as a compressed version of $\mathbf{x}$
- Some models map from data $\mathbf{x}$ to latent variable $\mathbf{z}$
 - e.g. **k-means algorithm** maps from $\mathbf{x}$ to cluster assignment $\mathbf{z}$ (cluster index).
 - e.g. **encoder** deep nets compress images to an embedding space $\mathbf{z}$
- Other models map from latent space $\mathbf{z}$ to data space $\mathbf{x}$
 - Distribution $Pr(\mathbf{z})$ allows to draw samples that can be mapped to data examples $\mathbf{x}$
 - Such models are termed **generative** models.
- Generative adversarial networks (GANs) learn to generate data examples $\mathbf{x}$ from latent variables $\mathbf{z}$
 - They are not probabilistic.

Taxonomy of Unsupervised Learning Models



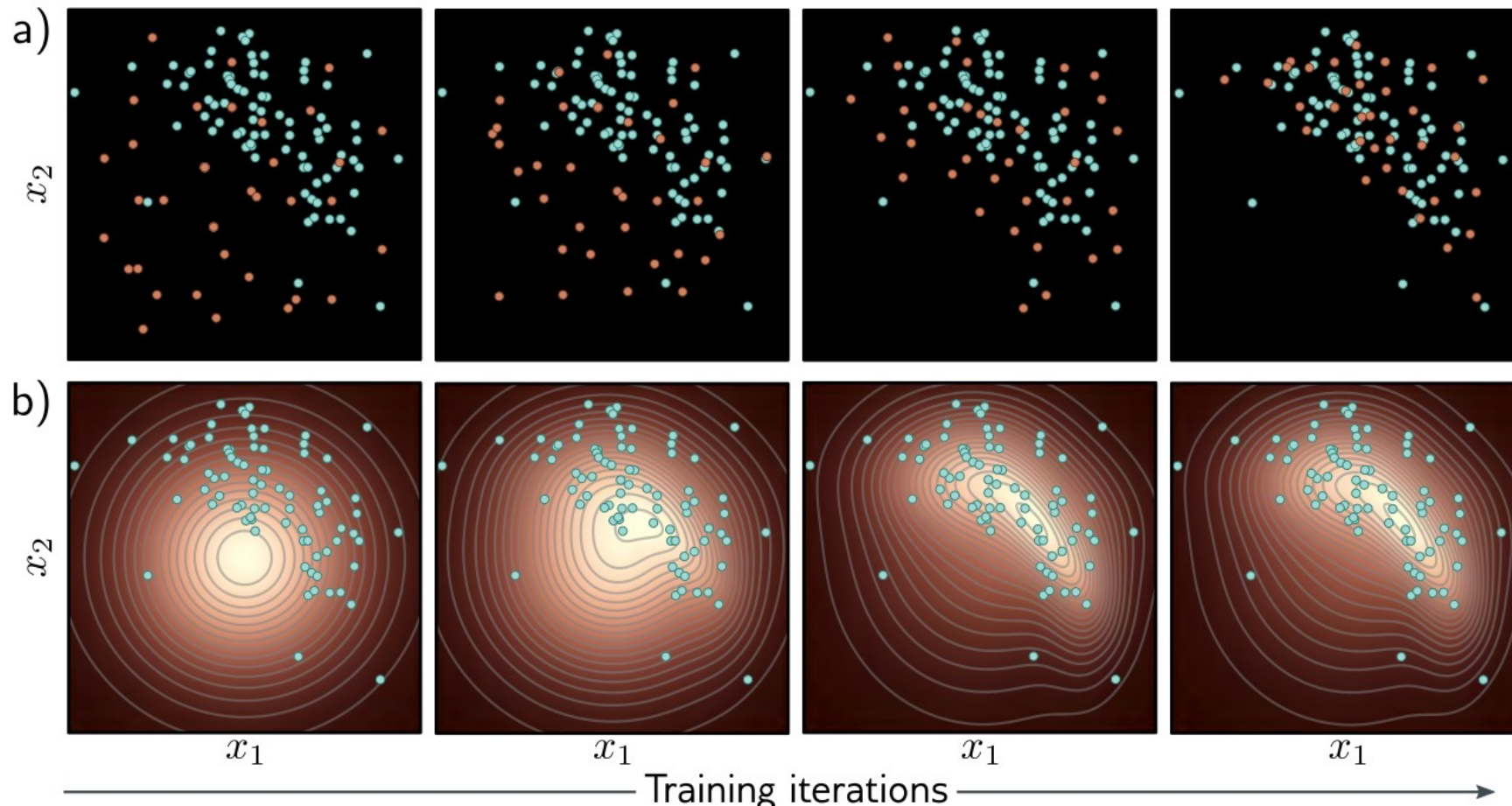
Probabilistic Generative Models

- Examples
 - Variational Auto-Encoder (VAE)
 - Diffusion Model
- Assign a probability $Pr(\mathbf{x}_i|\boldsymbol{\phi})$ to each data point $\mathbf{x}_i$ given model parameters $\boldsymbol{\phi}$
- We maximize the probability of the observed data.
So the loss is the sum of the negative log-likelihoods

$$L[\boldsymbol{\phi}] = - \sum_{i=1}^I \log \left[Pr(\mathbf{x}_i|\boldsymbol{\phi}) \right]$$

Fitting Generative Models

- a) GANs provide a mechanism for generating samples (orange points).
- b) Probabilistic models learn a probability over the training data.
 - As training proceeds the likelihood of the real examples increases under this distribution.

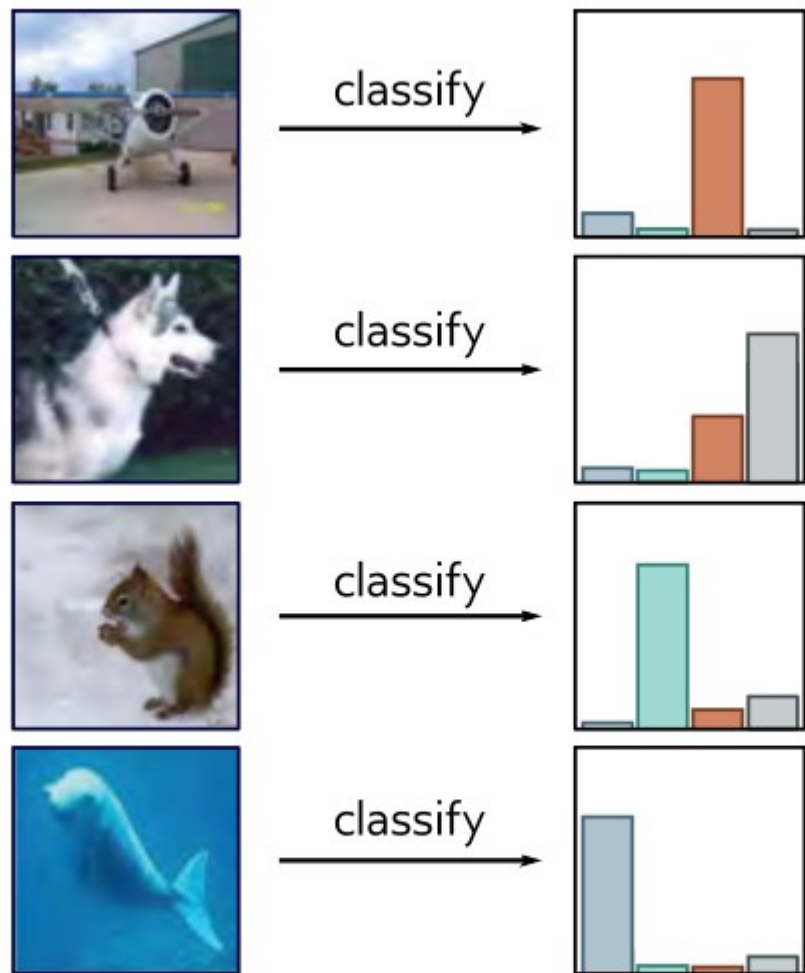


Quantifying Performance of Generative Models

- a) If generated images $\mathbf{x}^*$ are realistic, a trained deep-net classifier should output probabilities $Pr(y_i|\mathbf{x}^*)$ that are peaked at the correct class i . ($Pr(y_i \dots)$ is the classes/ i^{th} element of the one-hot output vector.)
- b) If the model generates all classes equally frequently, the marginal (average) class probability function $Pr(\mathbf{y})$ should be flat.

Quantifying Performance of Generative Models

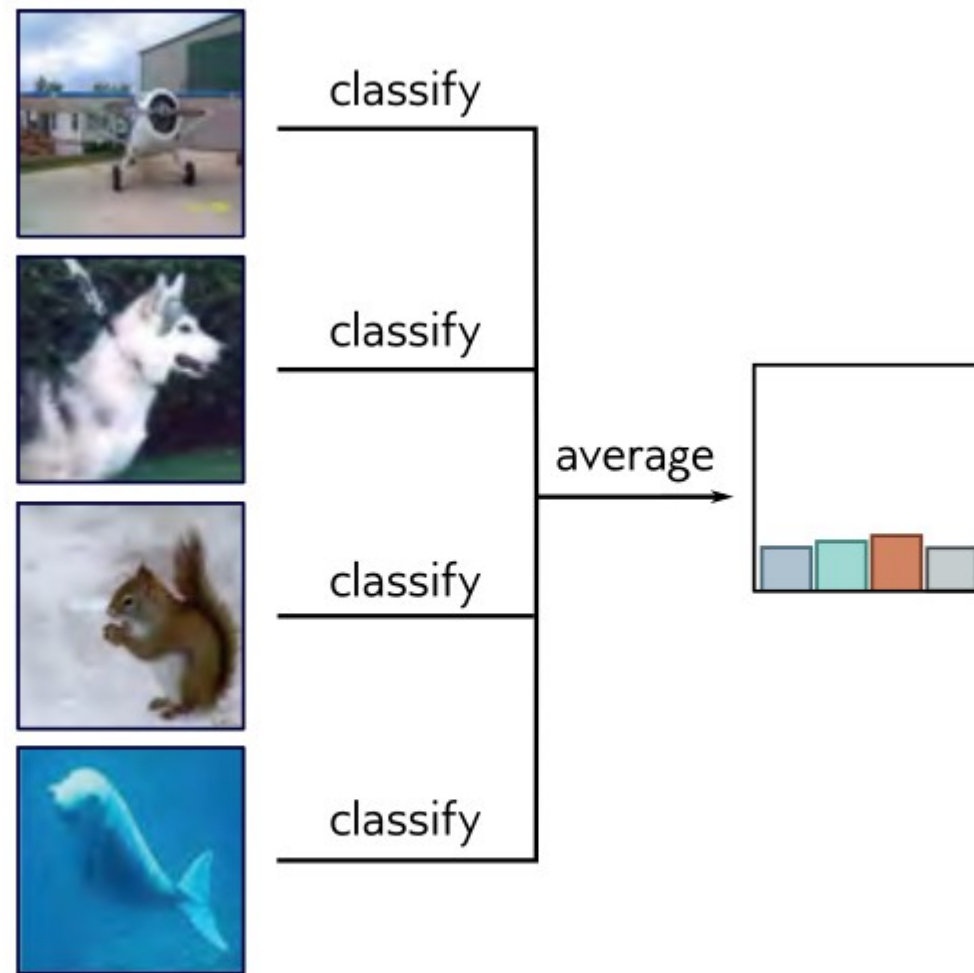
a)



Generated images
 $\{\mathbf{x}_i^*\}$

Classification
 $Pr(y_i|\mathbf{x}_i^*)$

b)



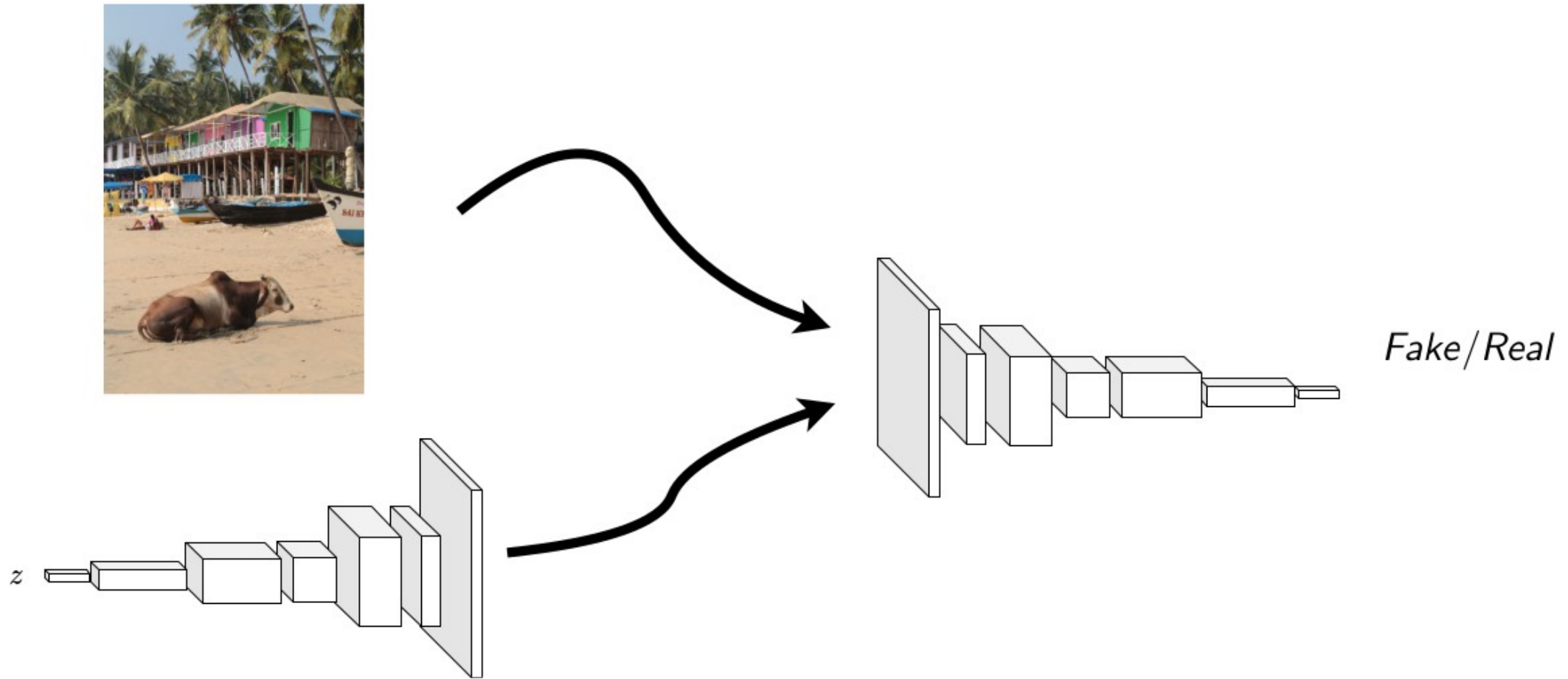
Generated images
 $\{\mathbf{x}_i^*\}$

Marginal class distribution
 $Pr(y) = \frac{1}{I} \sum_{i=1}^I Pr(y_i|\mathbf{x}_i^*)$

Generative Adversarial Networks

- A **GAN** is an **unsupervised model** that aims to **generate new samples** that are **indistinguishable** from a set of **training examples**.
- **Generator** network creates samples by mapping random noise $\mathbf{z}$ to the output data space $\mathbf{x}$.
- If a **discriminator** network **cannot distinguish** between generated samples and real examples, the **generator** does generate **plausible samples**.
- If the **discriminator can distinguish**, this provides a **training signal** for the **generator**.
- But training GANs is difficult.
- Even if the GAN generates indistinguishable samples, this **does not mean that all samples that are plausible can be generated**.

Generative Adversarial Networks

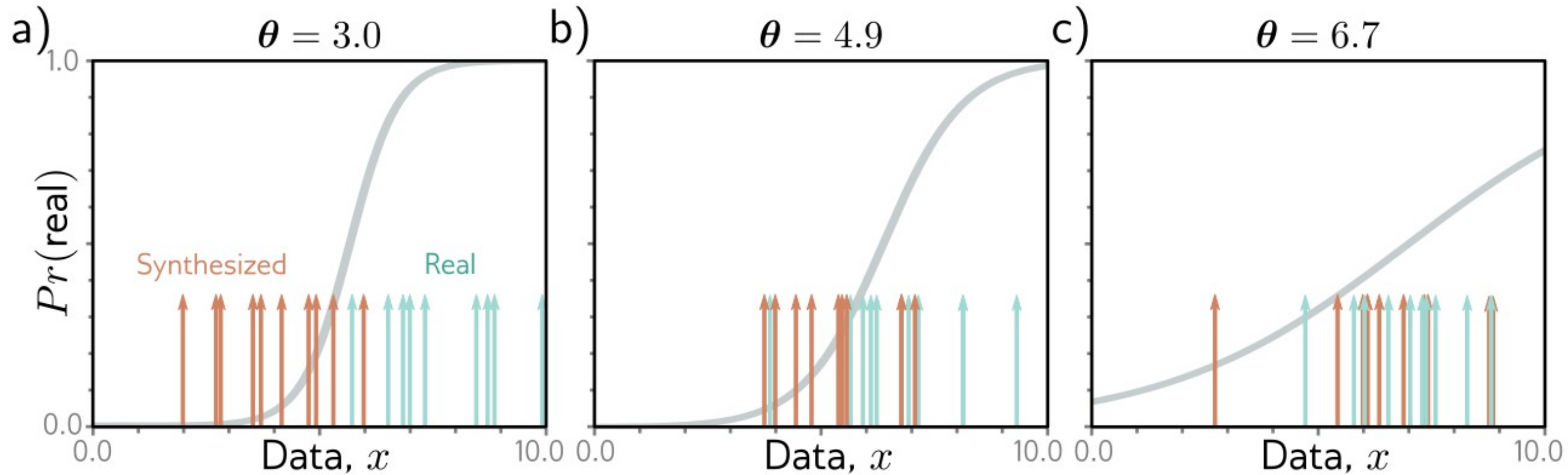


- Instead of generating an image from an input encoding, we can also just generate an image from a random vector.
- This way the generator learns to map the input distribution $p(z)$ to the data distribution $p(x)$.
- The discriminator learns to distinguish if an image x is within $p(x)$ or out of distribution.

Discrimination as a Signal

- Goal: Generate new samples $\{x^*_j\}$ that are drawn from the same distribution as a set of real training data $\{x_i\}$.
- Procedure:
 - a) Choose latent variable z_j from a simple base distribution (e.g. normal)
 - b) Pass it through a **generator network** $x^* = g[z_j, \theta]$ with parameters θ .
 - c) Find parameters θ so that the samples $\{x^*_j\}$ look “similar” to real data $\{x_i\}$.
Similarity: statistically indistinguishable from real data.
 - d) Discriminator network $f[x_i, \phi]$ classifies input as being real or generated.

Discrimination as a Signal



- Here latent variables are drawn from normal distribution.

$$x_j^* = g[z_j, \theta] = z_j + \theta$$

- Parameter θ translates generated samples along the x axis.

Binary Classification by the Discriminator

- Discriminator $f[x_i, \phi]$ does binary classification.
- Bernoulli distribution defined on domain $\{0, 1\}$ has a single parameter $\lambda \in [0, 1]$ that represents the probability that y takes value 1

$$Pr(y|\lambda) = \begin{cases} 1 - \lambda & y = 0 \\ \lambda & y = 1 \end{cases}$$

$$Pr(y|\lambda) = (1 - \lambda)^{1-y} \cdot \lambda^y$$

- Model $f[x_i, \phi]$ predicts λ . It has to lie in range $[0, 1]$.

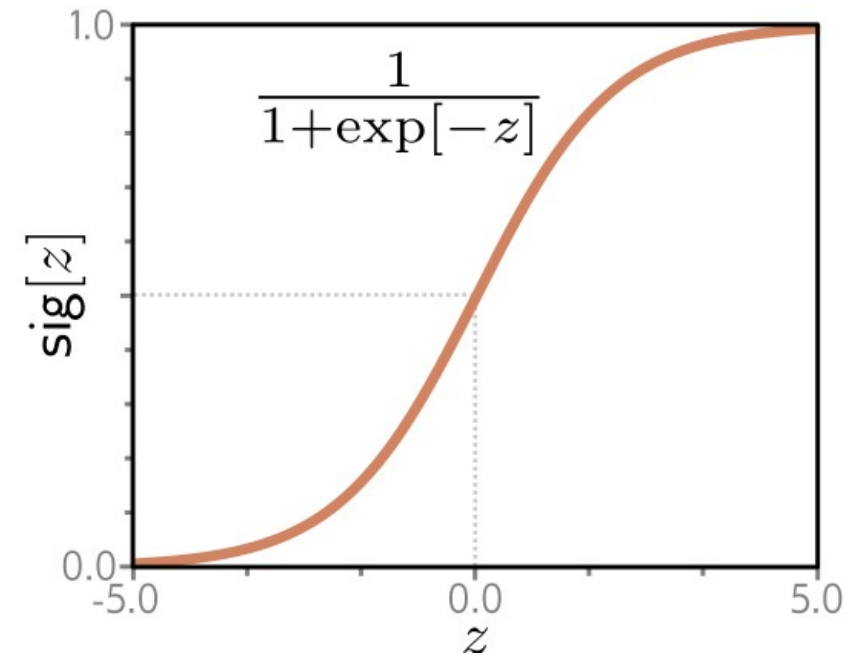
Binary Classification

- So network output is passed through function mapping real number to $[0, 1]$.

$$\text{sig}[z] = \frac{1}{1 + \exp[-z]} \quad \lambda = \text{sig}[f[\mathbf{x}, \phi]]$$

- The likelihood λ is now:

$$Pr(y|\mathbf{x}) = (1 - \text{sig}[f[\mathbf{x}, \phi]])^{1-y} \cdot \text{sig}[f[\mathbf{x}, \phi]]^y$$



- Loss function is the **negative log-likelihood** of the training set:

$$L[\phi] = \sum_{i=1}^I -(1 - y_i) \log[1 - \text{sig}[f[\mathbf{x}_i, \phi]]] - y_i \log[\text{sig}[f[\mathbf{x}_i, \phi]]]$$

- This can be shown to be the binary **cross-entropy loss**.

Cross-Entropy Loss

- Idea: Find parameters that minimize the distance between **empirical/real distribution** of observed data and **model distribution $\Pr(\mathbf{y}|\boldsymbol{\theta})$** (our knowledge about reality).
- Distance between two probability distributions can be evaluated using the Kullback-Leibler (KL) divergence.

$$D_{KL}[q||p] = \int_{-\infty}^{\infty} q(z) \log[q(z)] dz - \int_{-\infty}^{\infty} q(z) \log[p(z)] dz$$

- We want to minimize w.r.t. the model distribution parameters $\boldsymbol{\theta}$:

$$\begin{aligned}\hat{\boldsymbol{\theta}} &= \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \left[\int_{-\infty}^{\infty} q(y) \log[q(y)] dy - \int_{-\infty}^{\infty} q(y) \log[\Pr(y|\boldsymbol{\theta})] dy \right] \\ &= \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \left[- \int_{-\infty}^{\infty} q(y) \log[\Pr(y|\boldsymbol{\theta})] dy \right],\end{aligned}$$

Cross-Entropy Loss

- We sample at the positions of the observed data:

$$\begin{aligned}\hat{\boldsymbol{\theta}} &= \operatorname{argmin}_{\boldsymbol{\theta}} \left[- \int_{-\infty}^{\infty} \left(\frac{1}{I} \sum_{i=1}^I \delta[y - y_i] \right) \log[Pr(y|\boldsymbol{\theta})] dy \right] \\ &= \operatorname{argmin}_{\boldsymbol{\theta}} \left[- \frac{1}{I} \sum_{i=1}^I \log[Pr(y_i|\boldsymbol{\theta})] \right] \\ &= \operatorname{argmin}_{\boldsymbol{\theta}} \left[- \sum_{i=1}^I \log[Pr(y_i|\boldsymbol{\theta})] \right].\end{aligned}$$

- The distribution parameters are computed by the model

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[- \sum_{i=1}^I \log[Pr(y_i|\mathbf{f}[\mathbf{x}_i, \phi])] \right]$$

which is the negative log-likelihood from before.

GAN Loss Function

- Original form

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[\sum_i -(1 - y_i) \log \left[1 - \operatorname{sig}[f[\mathbf{x}_i, \phi]] \right] - y_i \log \left[\operatorname{sig}[f[\mathbf{x}_i, \phi]] \right] \right]$$

- With $y=1$ for real examples $\mathbf{x}$,
and $y=0$ for generated samples $\mathbf{x}^*$

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[\sum_j -\log \left[1 - \operatorname{sig}[f[\mathbf{x}_j^*, \phi]] \right] - \sum_i \log \left[\operatorname{sig}[f[\mathbf{x}_i, \phi]] \right] \right]$$

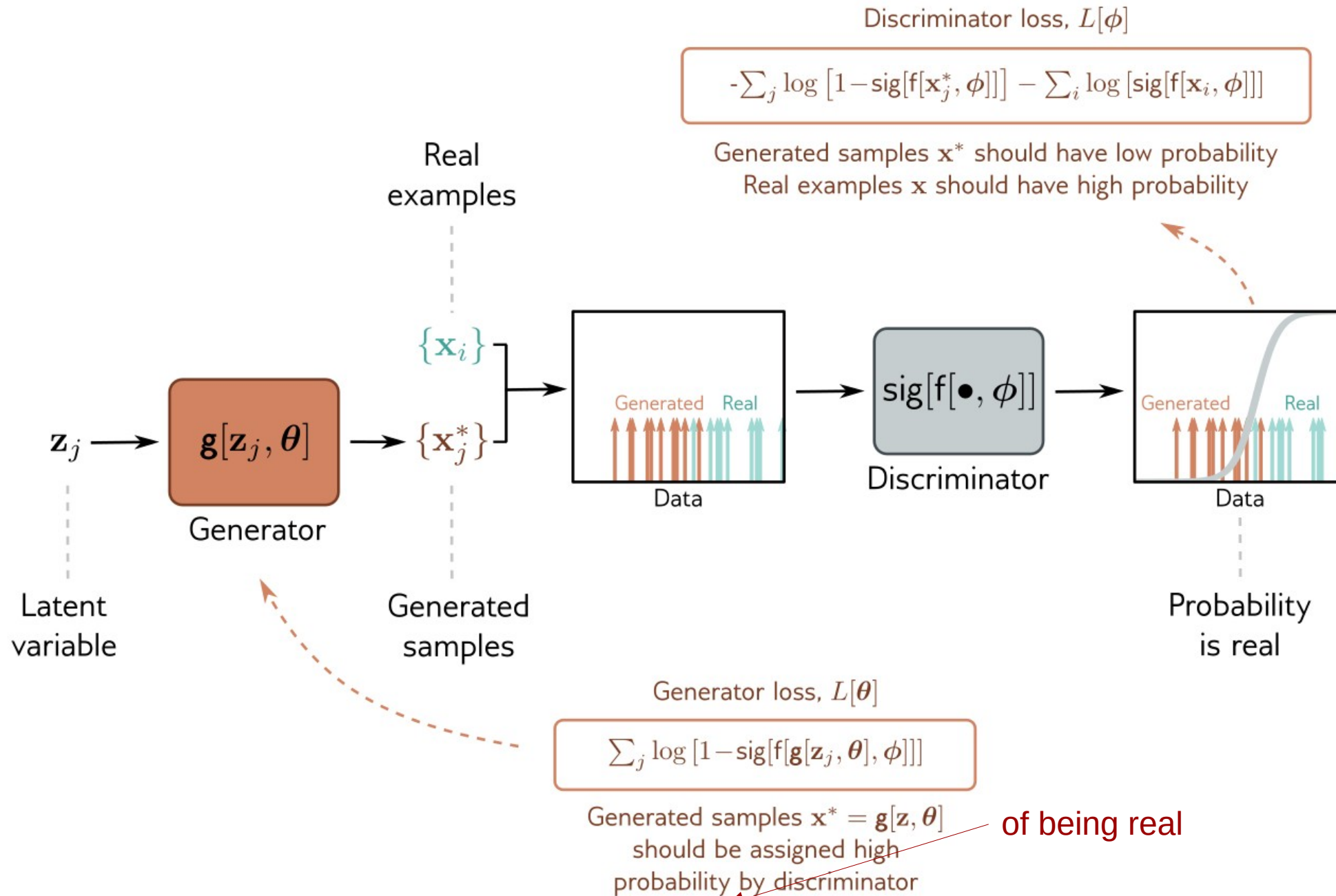
GAN Loss Function

- Now we substitute the definition for the generator $x^* = g[z_j, \theta]$.
- As we want the generated samples to be misclassified (have low likelihood of being synthetic or high negative log-l.), we **maximize** w.r.t. θ :

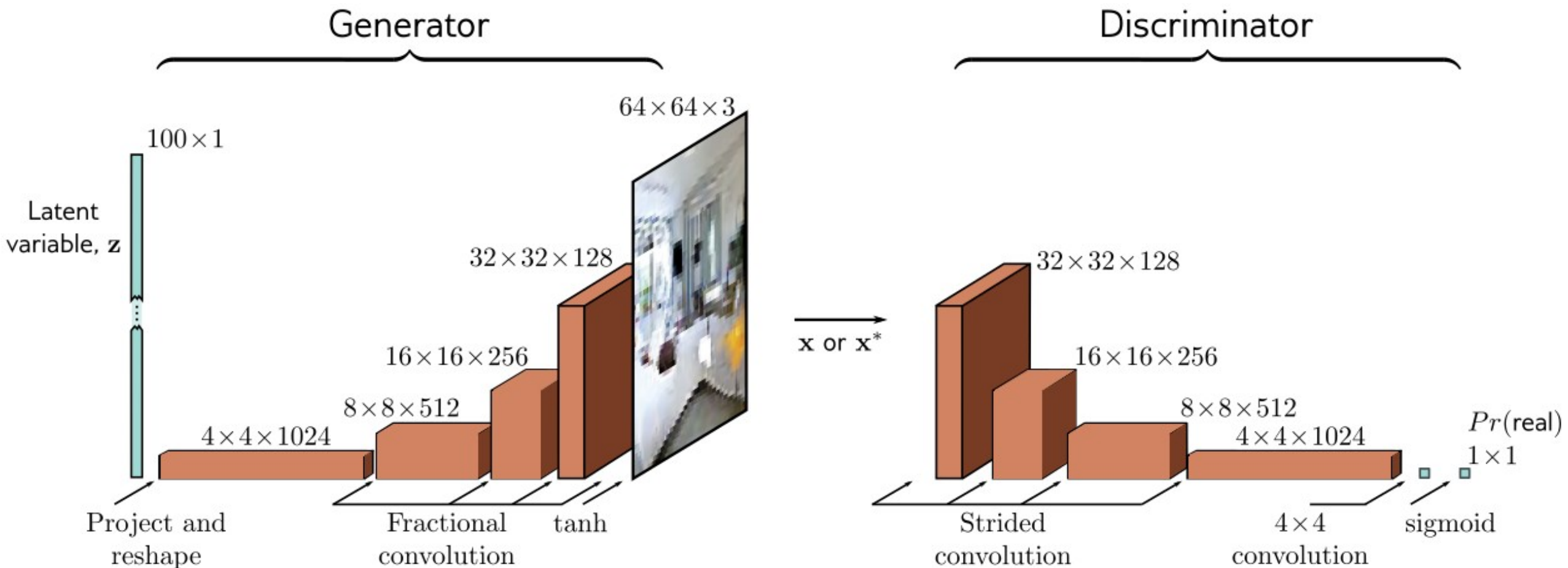
$$\hat{\theta} = \operatorname{argmax}_{\theta} \left[\min_{\phi} \left[\sum_j -\log \left[1 - \operatorname{sig}[f[g[z_j, \theta], \phi]] \right] - \sum_i \log \left[\operatorname{sig}[f[x_i, \phi]] \right] \right] \right]$$

- More complex loss function than other models
 - 1) Discriminator parameters manipulated to minimize loss function
 - 2) Generator parameters manipulated to maximize loss function→ minimax game
- Upon convergence **generator draws from real distribution**, and **discriminator guesses at chance 0.5**.

GAN Loss Function



Deep Convolutional GAN (DCGAN)

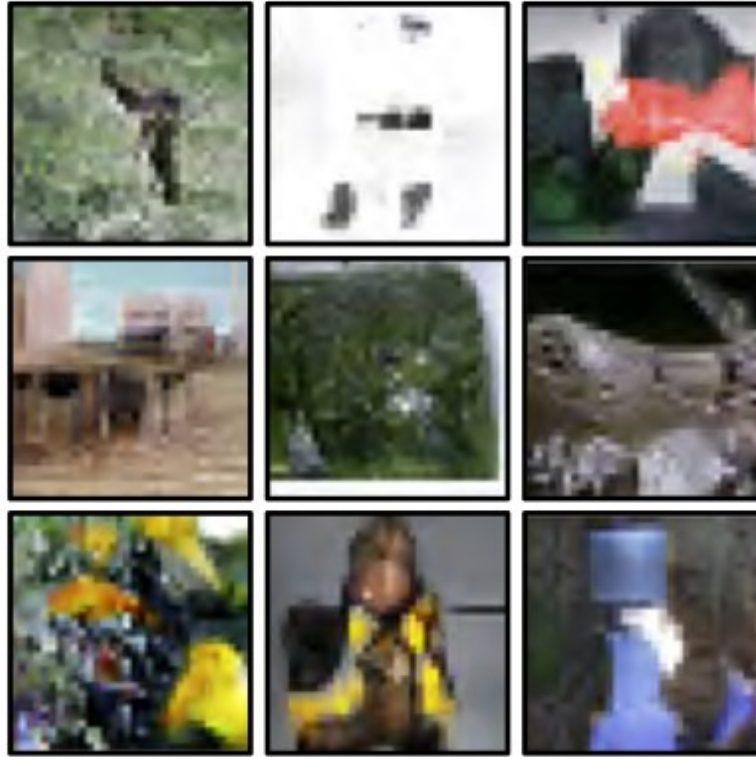


Deep Convolutional GAN (DCGAN)

a)



b)



c)

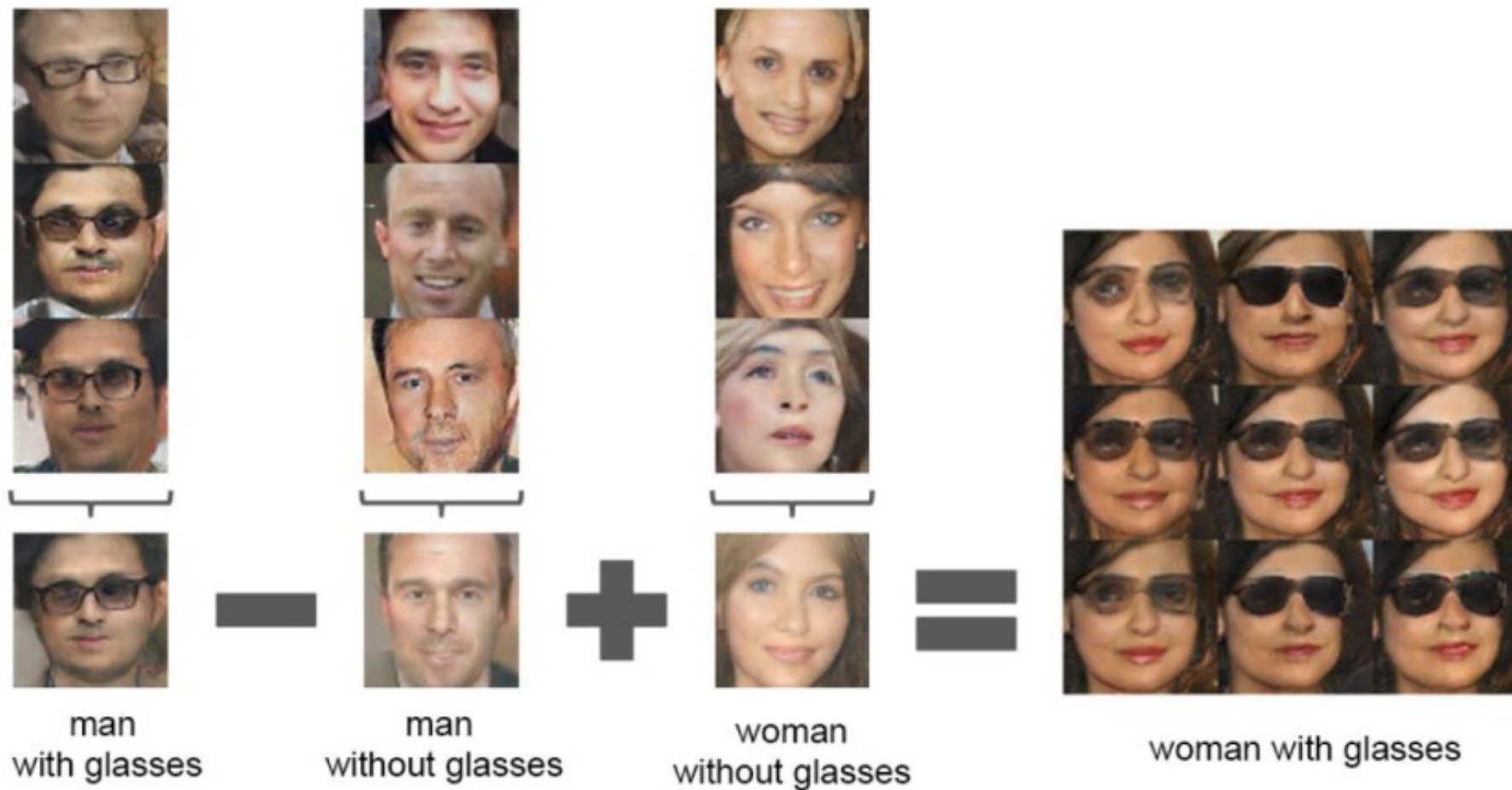


a) Random samples drawn from DCGAN trained on face dataset

b) Random samples using ImageNet database

c) Random samples drawn from LSUN scene understanding.

Deep Convolutional GANs



- Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks, Radford et al., ICLR 2016
- Paper also uses discriminator features for image classification and lists design guidelines for ConvNet architectures for GANs.

Difficulty Training GANs

- GANs are notoriously difficult to train.
- To be successful, DCGAN required special measures
 - i. Strided convolutions for up- and downsampling
 - ii. BatchNorm in generator and discriminator
 - iii. Leaky ReLU activation function
 - iv. Adam optimizer
- Problems
 - a) Generated samples represent subset of the data only.
→ **Mode Dropping**
 - b) Generator even ignores latent variables z and collapses all samples to one.
→ **Mode Collapse**

MLP GAN (instead of DCGAN)



Mode collapse:

- Synthesized images from a GAN trained on LSUN dataset using an MLP generator with a similar number of parameters and layers compared to DCGAN

Analysis of GAN Loss Function

- We have seen that GAN loss can be interpreted in terms of **distances between probability distributions**.
- Mathematical analysis shows that regions with samples x^* but no real examples x are penalized.
 - I.e. the loss function (used so far) enforces quality.
- Mathematical analysis shows that regions with real examples but no samples are penalized.
 - i.e. the loss function enforces coverage.
- **Problem remains:**
Small changes of the probability function of generator towards probability function of real data are difficult to obtain.

Wasserstein Distance

- There is a need for a better loss function.
- **Wasserstein distance** between two probability functions:
Cost associated with moving probability mass in order to **reshape the first probability function** such that it gets the shape of the second probability function
- **Wasserstein loss** function makes GAN training more stable.

Wasserstein Distance D_w

- Also known as **Earth Mover's Distance**
- The “amounts/masses” moved from the transport plan stored in (transport-plan) matrix **P**.

- Discrete formulation
$$D_w[Pr(x)||q(x)] = \min_{\mathbf{P}} \left[\sum_{i,j} P_{ij} \cdot |i - j| \right] ,$$

subject to the constraints that:

$$\sum_j P_{ij} = Pr(x = i)$$

$$\sum_i P_{ij} = q(x = j)$$

$$P_{ij} \geq 0$$

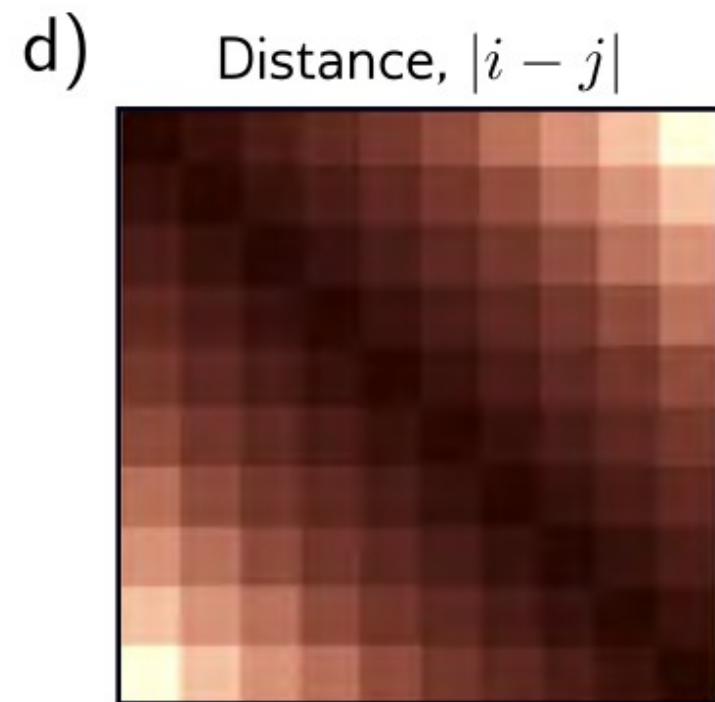
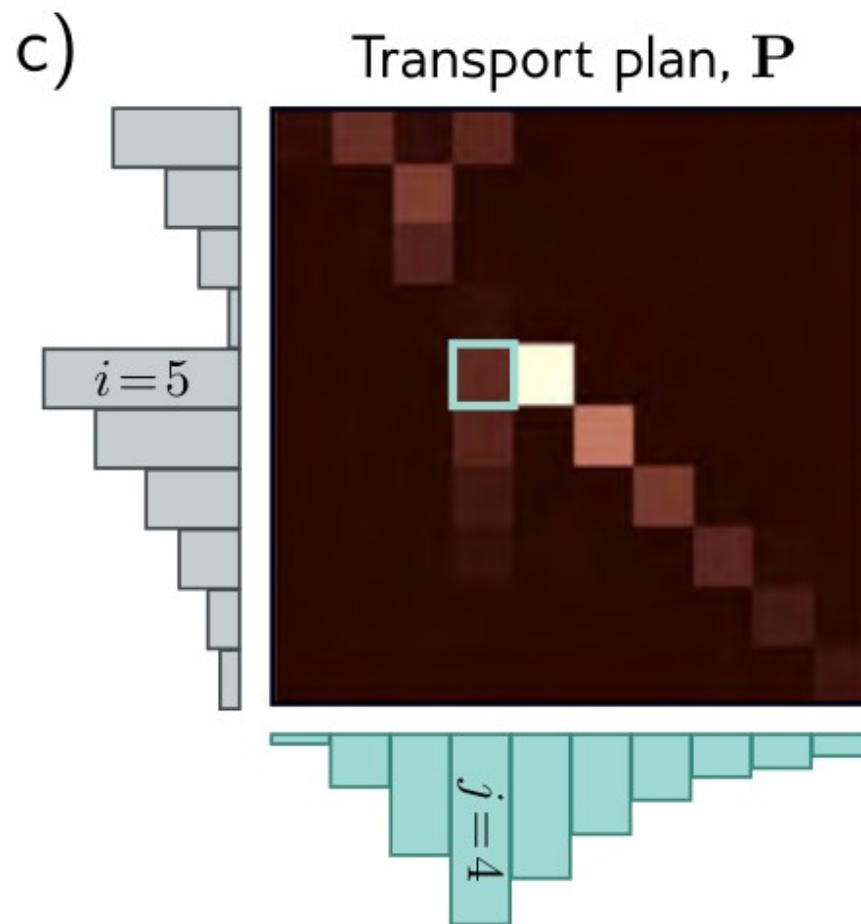
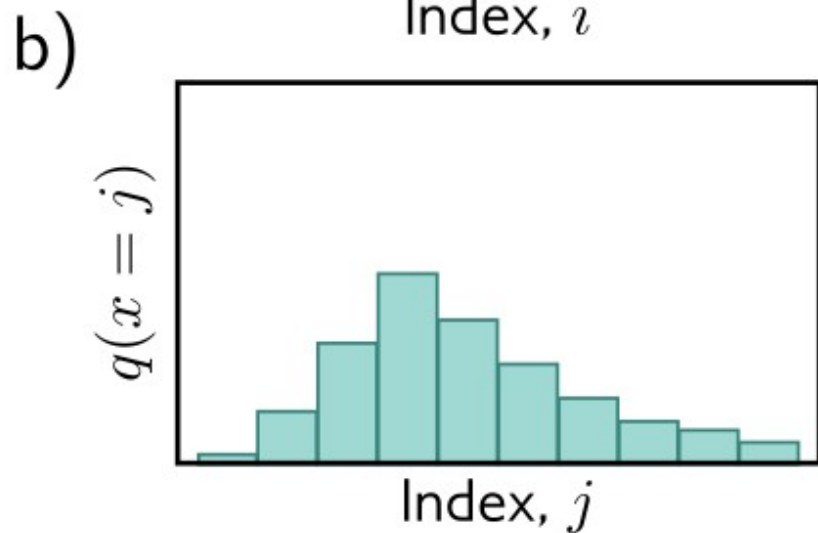
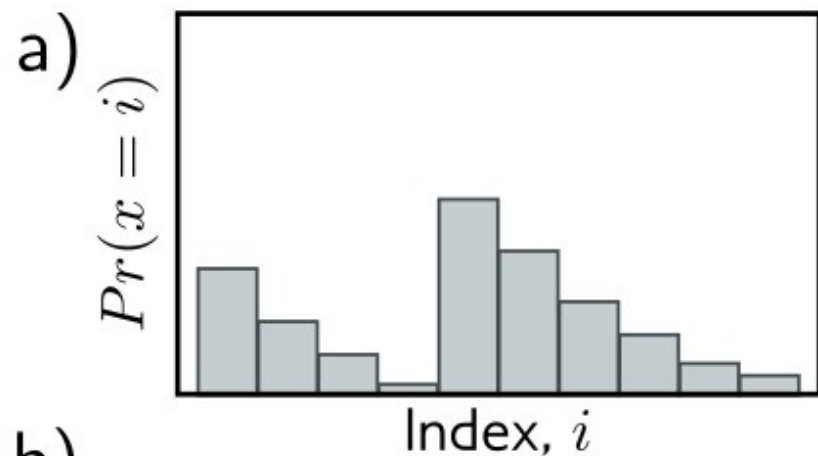
initial distribution of $Pr(x)$

initial distribution of $q(x)$

non-negative masses.

- This is a **linear programming** problem.

Wasserstein Distance



Wasserstein distance
 $= \sum \mathbf{P} \cdot |i-j|$

Wasserstein Distance for Continuous Functions

$$D_w[Pr(\mathbf{x}), q(\mathbf{x})] = \min_{\pi[\bullet, \bullet]} \left[\iint \pi(\mathbf{x}_1, \mathbf{x}_2) \cdot ||\mathbf{x}_1 - \mathbf{x}_2|| d\mathbf{x}_1 d\mathbf{x}_2 \right]$$

- Transport plan $\pi(\mathbf{x}_1, \mathbf{x}_2)$ representing the mass moved from position $\mathbf{x}_1$ to $\mathbf{x}_2$.

- Linear programming solution

$$D_w[Pr(\mathbf{x}), q(\mathbf{x})] = \max_{f[\mathbf{x}]} \left[\int Pr(\mathbf{x}) f[\mathbf{x}] d\mathbf{x} - \int q(\mathbf{x}) f[\mathbf{x}] d\mathbf{x} \right]$$

- Optimization over space of functions $f[\mathbf{x}]$ by optimizing parameters ϕ in a neural network

$$L(\phi) = \frac{1}{m_g} \sum_{j=1}^{m_g} f(x_j^*, \phi) - \frac{1}{m_r} \sum_{i=1}^{m_r} f(x_i, \phi)$$

Difference of average potentials;
 f fulfilling 1-Lipshitz property
 (see slide 43);

f is not a probability but only a scalar.

GAN Cross-Entropy Loss Function

$$\hat{\theta} = \operatorname{argmax}_{\theta} \left[\min_{\phi} \left[\sum_j -\log \left[1 - \operatorname{sig}[f[\mathbf{g}[\mathbf{z}_j, \theta], \phi]] \right] - \sum_i \log \left[\operatorname{sig}[f[\mathbf{x}_i, \phi]] \right] \right] \right]$$

- The GAN loss function can be divided into two functions:

$$L[\phi] = \sum_j -\log \left[1 - \operatorname{sig}[f[\mathbf{g}[\mathbf{z}_j, \theta], \phi]] \right] - \sum_i \log \left[\operatorname{sig}[f[\mathbf{x}_i, \phi]] \right]$$

$$L[\theta] = \sum_j \log \left[1 - \operatorname{sig}[f[\mathbf{g}[\mathbf{z}_j, \theta], \phi]] \right]$$

- f is the discriminator network (as shown before).

Wasserstein Distance Loss Function

$$\begin{aligned} L[\phi] &= \sum_j f[\mathbf{x}_j^*, \phi] - \sum_i f[\mathbf{x}_i, \phi] \\ &= \sum_j f[\mathbf{g}[\mathbf{z}_j, \boldsymbol{\theta}], \phi] - \sum_i f[\mathbf{x}_i, \phi] \end{aligned} \quad \text{with} \quad \left| \frac{\partial f[\mathbf{x}, \phi]}{\partial \mathbf{x}} \right| < 1$$

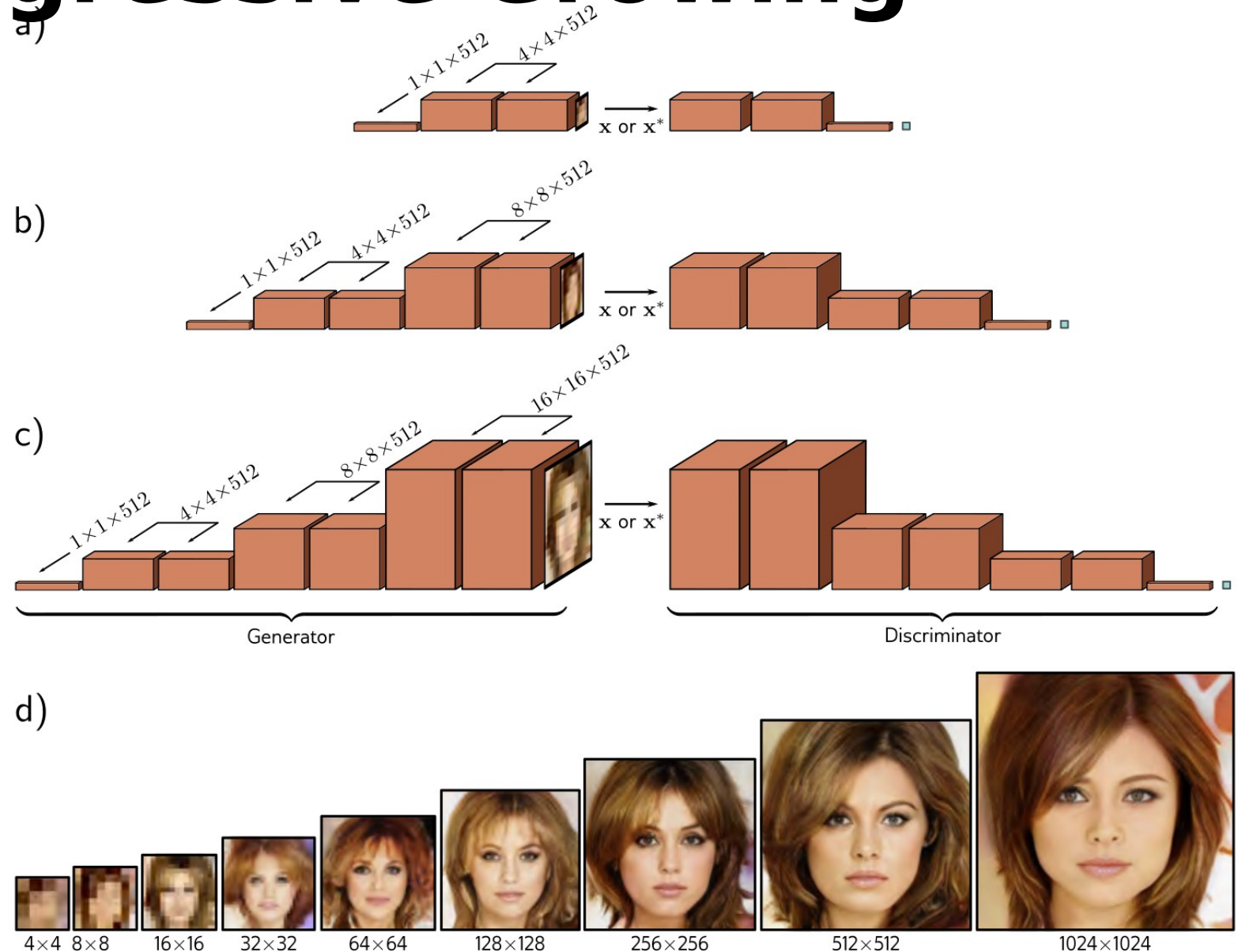
- f is analogous to the discriminator but without the sigmoid activation, and it is trained to approximate the Wasserstein distance.
- Always provides meaningful gradients.
- Reduces the likelihood of mode collapse.
- Loss correlates with the quality of the generated samples.
- 1-Lipschitz $\left| \frac{\partial f[\mathbf{x}, \phi]}{\partial \mathbf{x}} \right| < 1$ gradient ensures controlled training.
Gradient not steeper than 45°

Wasserstein Distance Loss Function

- Comment:
 - WGAN improves the geometry of GAN training because Wasserstein distance gives more meaningful gradients than cross-entropy.
 - However, if the ground metric is pixel Euclidean distance, the resulting Wasserstein geometry is still not identical to human perceptual geometry.
 - This mismatch is one reason WGAN losses do not perfectly track visual quality and one reason training still requires care.

Further Improvements: Progressive Growing

- An initially 4x4 image is **upsampled** in additional layers.
- The discriminator has these added layers, too.
- **Higher-resolution information “fades in”** over several iterations.

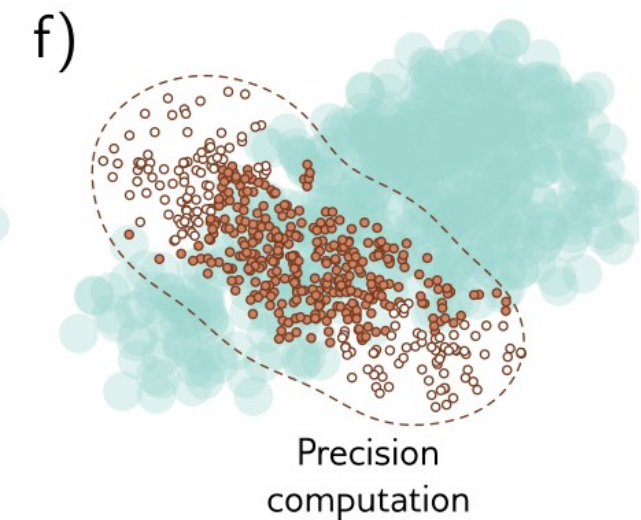
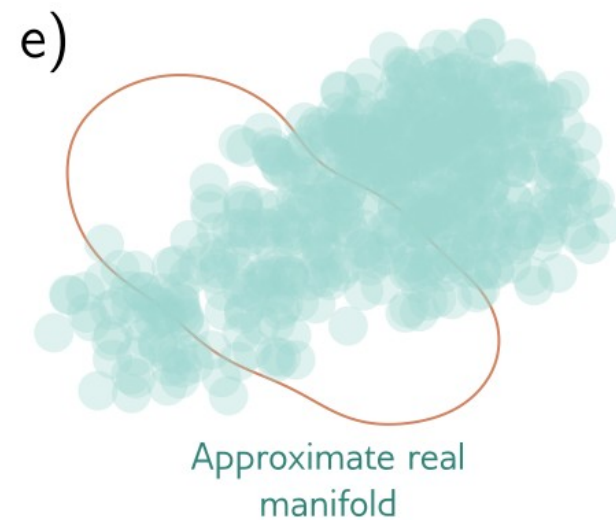
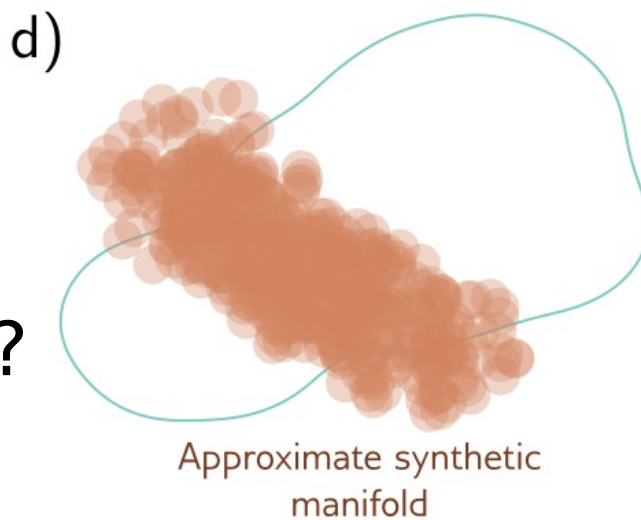
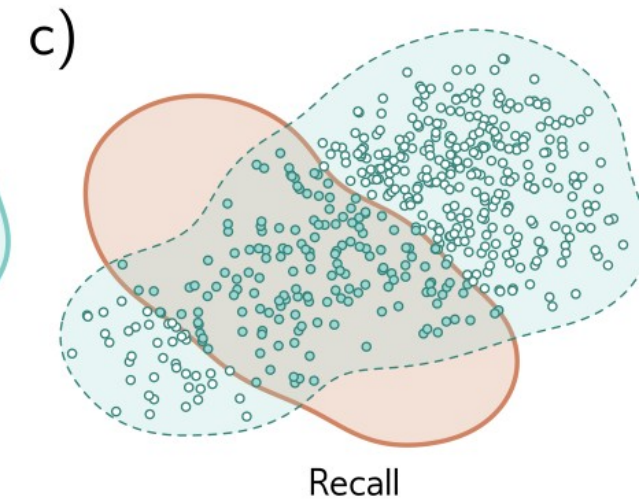
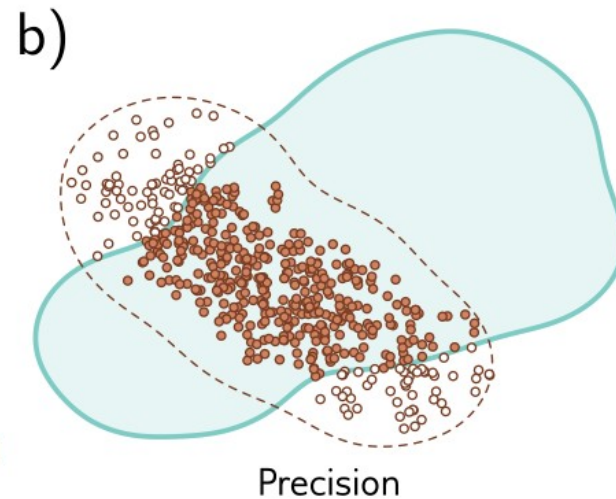
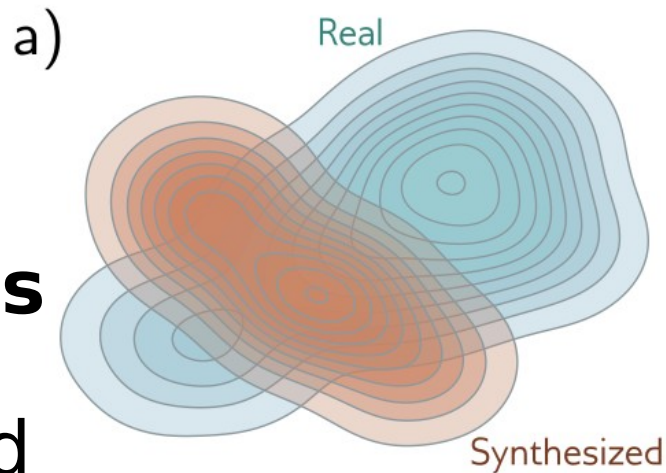


Further Improvements: **Mini-Batch Discrimination**

- Discriminator computes **feature statistics** for **synthesized** and **real** data across mini batches.
- Feature statistics are added to the **discriminator signal** that is used to **train the generator**.
- Generator is encouraged to **generate images** having the **same variety as the real images**.

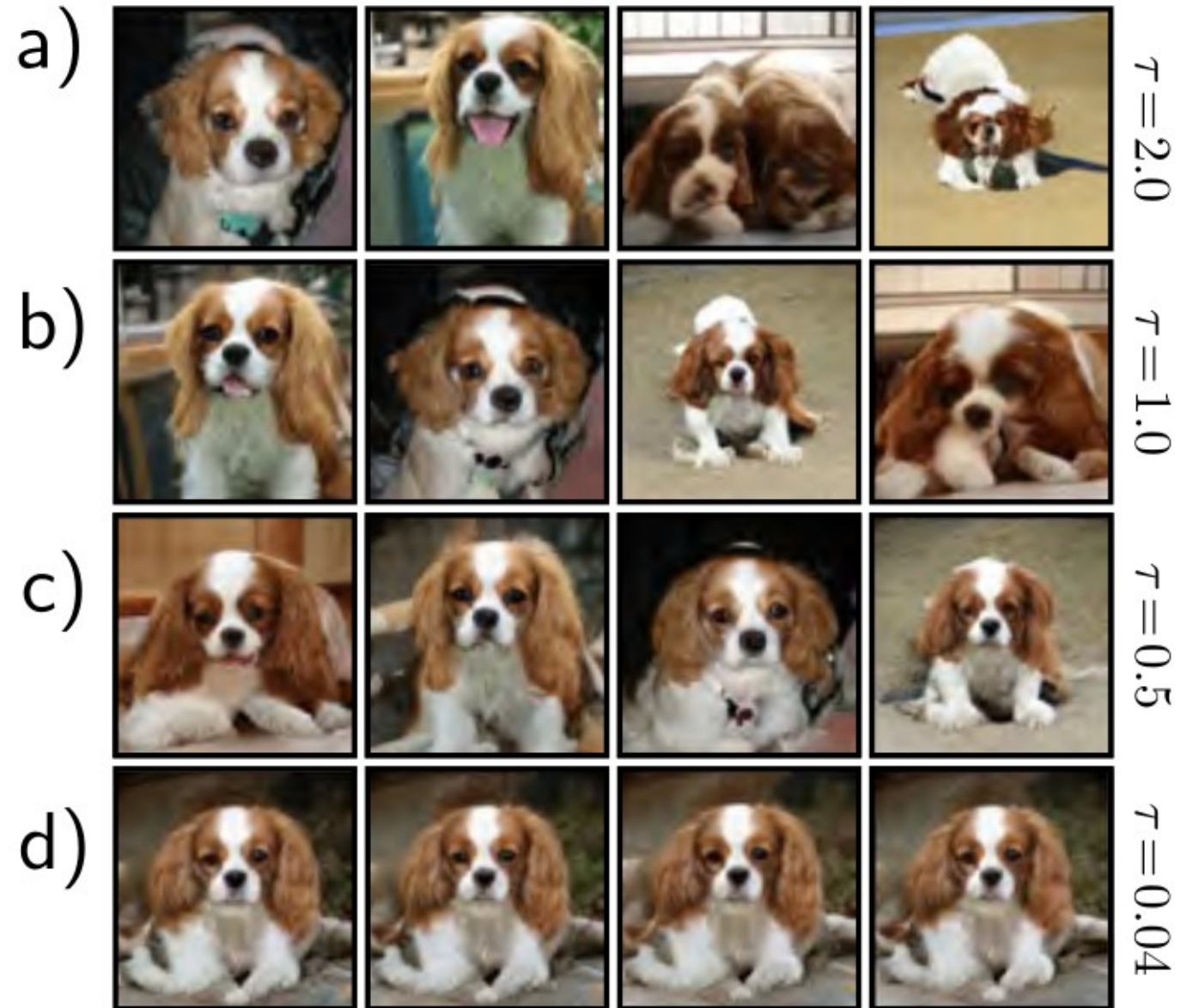
Further Improvements: Manifold Precision/Recall

- Investigating the **true/empirical distributions** of real examples and synthesized samples
- In which space?
Latent space?



Further Improvements: **Truncation**

- **Quality** of the generated samples can be traded off **against diversity** by rejecting samples from the latent variable $\mathbf{z}$ that fall further than τ **standard deviations** from the mean.



Further Improvements

- Using **combinations of the above methods**
GANs can synthesize varied and realistic images.
- E.g. training on selected training datasets



Further Improvements

- Moving **smoothly through latent space** can produce **realistic interpolations**.

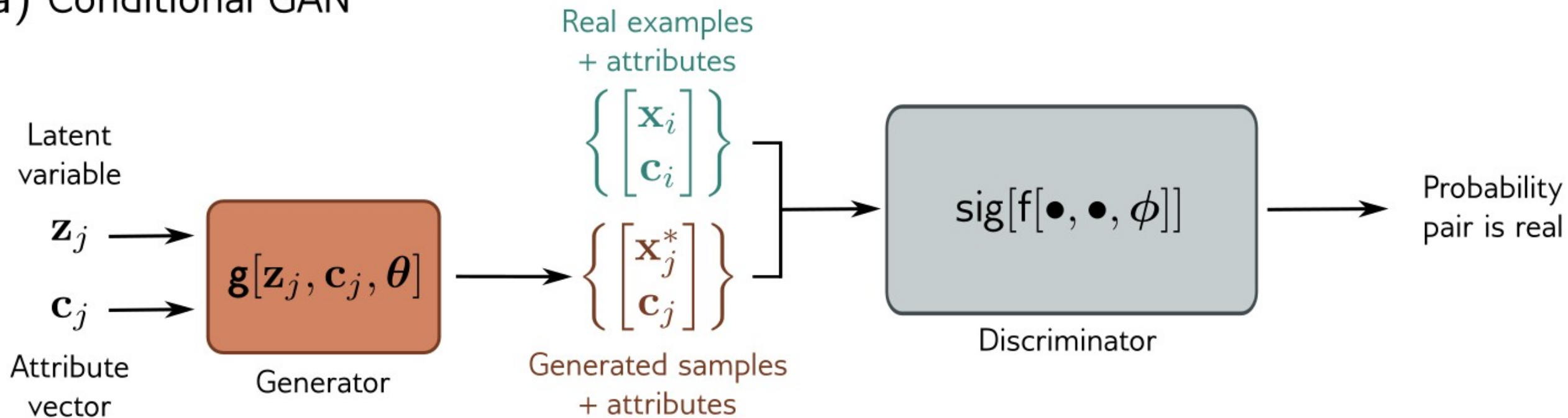


Conditional Generation: Conditional GAN

- The GANs we have seen so far cannot produce images of a specified kind.
- This can be achieved by a **Conditional GAN** that passes an attribute vector $\mathbf{c}$ to both generator $g[\mathbf{z}, \mathbf{c}, \boldsymbol{\theta}]$ and discriminator $f[\mathbf{x}, \mathbf{c}, \phi]$.
- For the **generator** $\mathbf{c}$ can be appended to $\mathbf{z}$.
- For the **discriminator** $\mathbf{c}$ can first be **linearly transformed to a 2D representation** and then added as an **extra channel of the input image**.

Conditional GAN

a) Conditional GAN



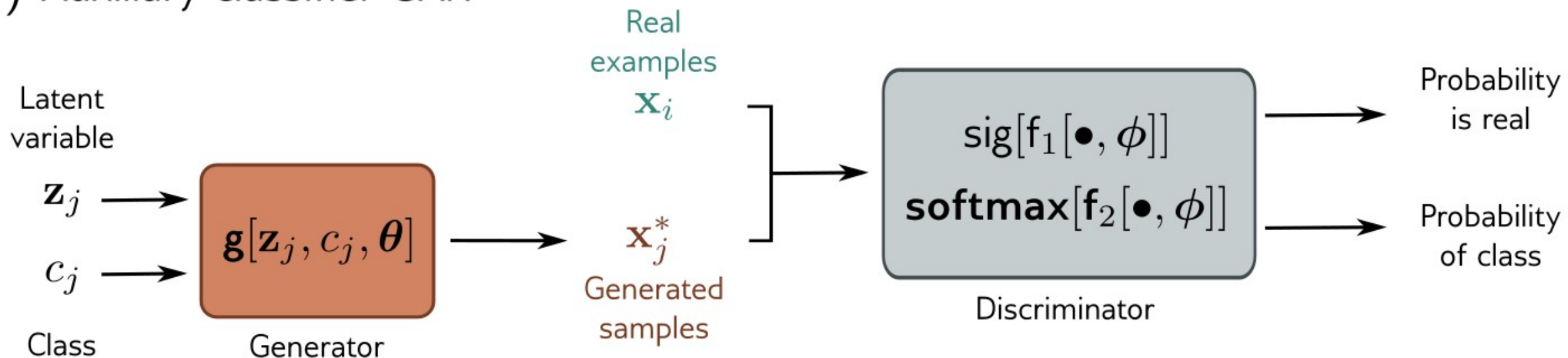
Conditional Generation: Auxiliary Classifier GAN

- ACGAN simplifies conditional generation by requiring that the **discriminator correctly predicts the attribute**.
- For a discrete attribute with C categories, the discriminator takes the image as input and has $C+1$ outputs.
- Networks trained with ACGAN can synthesize multiple classes, e.g. from ImageNet.



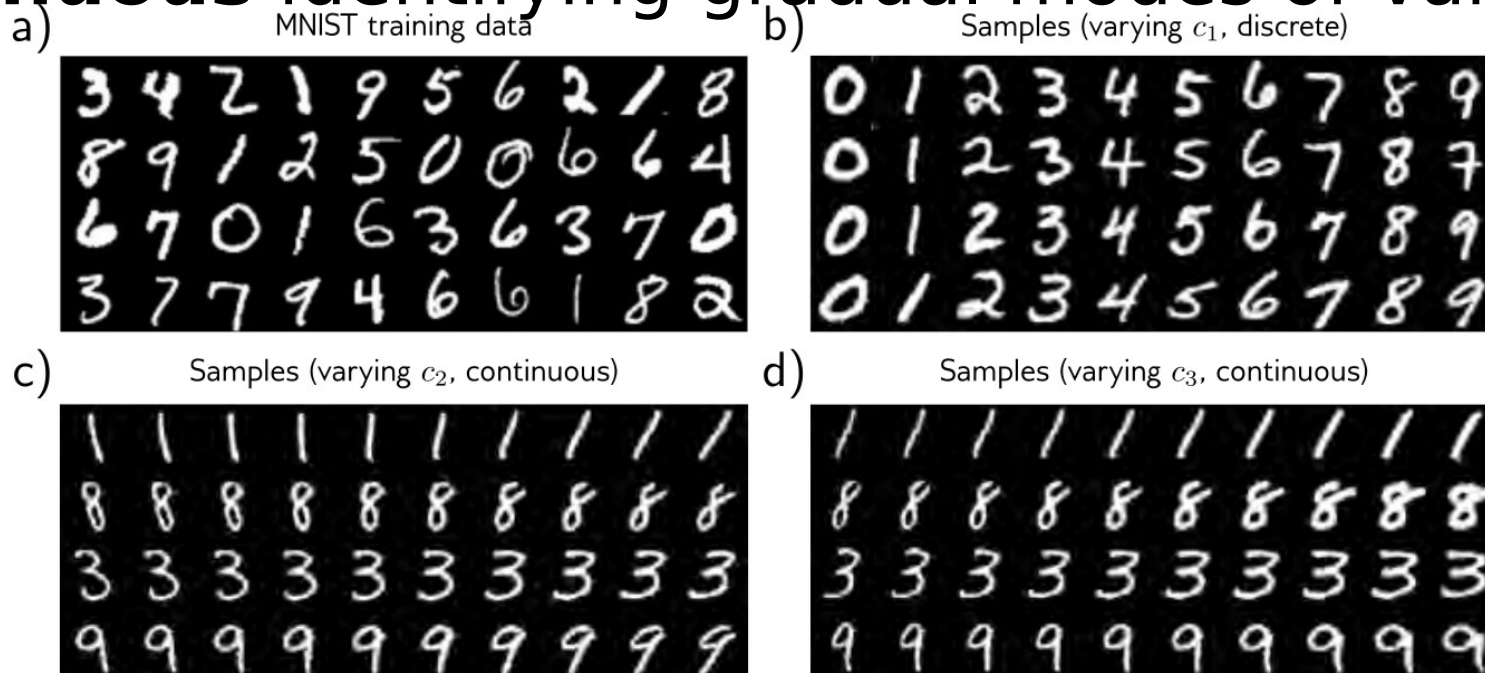
Auxiliary Classifier GAN

b) Auxiliary classifier GAN



Conditional Generation: InfoGAN

- attempts to identify attributes automatically.
- Generator takes random noise variable $\mathbf{z}$ and random attribute variable $\mathbf{c}$.
- Attributes may be **discrete**, e.g. classes,
- or **continuous** identifying gradual modes of variation



InfoGAN

c) InfoGAN

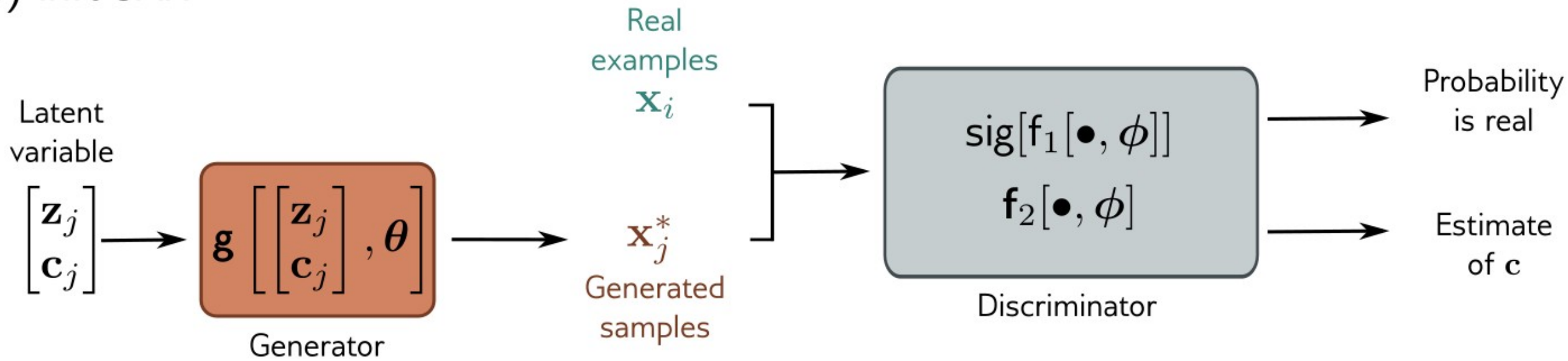


Image Translation

- Although the adversarial discriminator was originally used in the context of the GAN for **generating random samples**, it can also be used as a prior that favors **realism in translating one data example into another**.

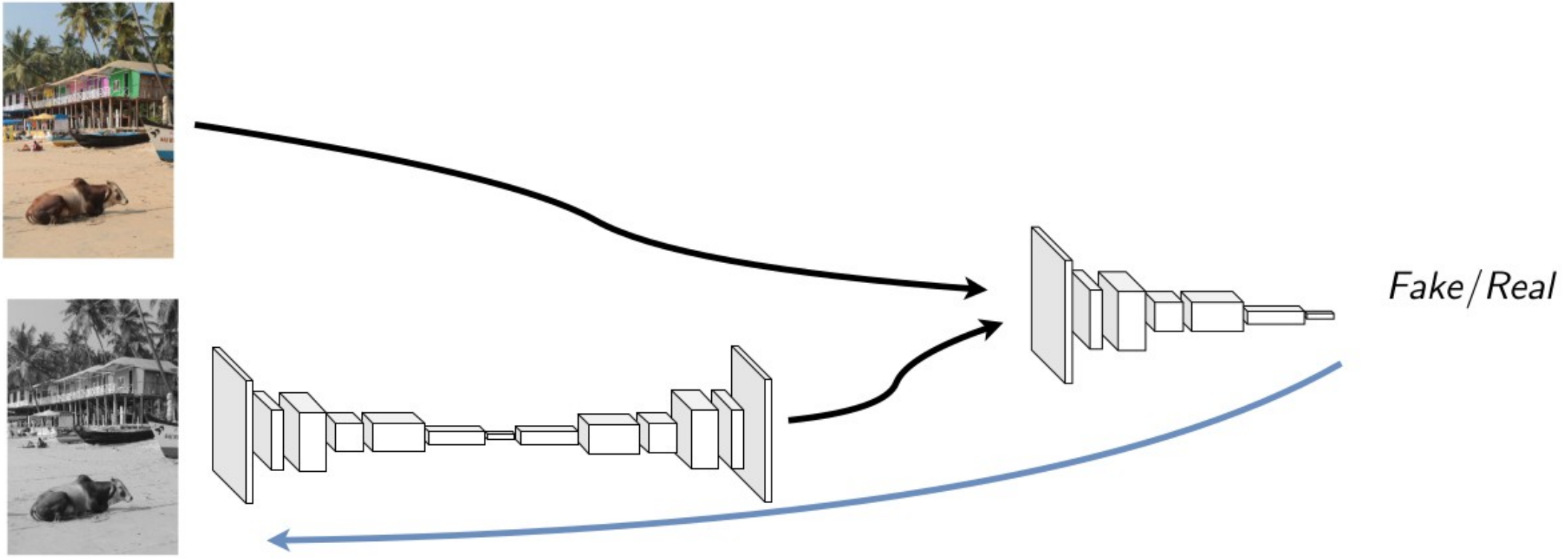


Image Translation: Pix2Pix

- Maps one image $\mathbf{c}$ to a different style image $\mathbf{x}$ using a U-Net with parameters θ .
- E.g. colorization
- Content loss: l_1 norm $\|\mathbf{x} - g[\mathbf{c}, \theta]\|_1$
- Adversarial discriminator $f[\mathbf{c}, \mathbf{x}, \phi]$
with before image $\mathbf{x}$ and after image $\mathbf{z}$ as input
tries to distinguish **real before/after pair** from **before/synthesized pair**.
Feedback signal used to train the generator (as usual for GANs)
- C.f. $\mathbf{x}$ as the condition $\mathbf{c}$ in a Conditional GAN.

Image Translation: Pix2Pix

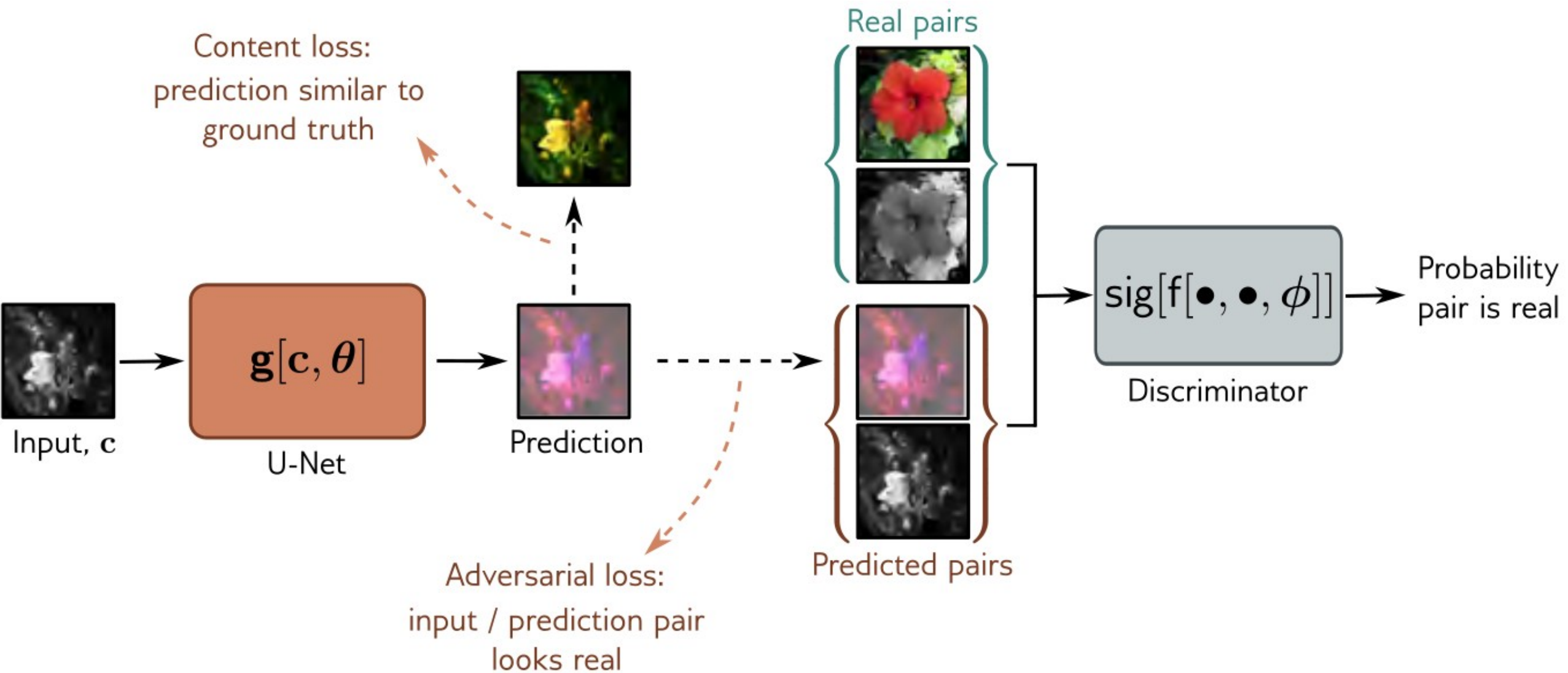
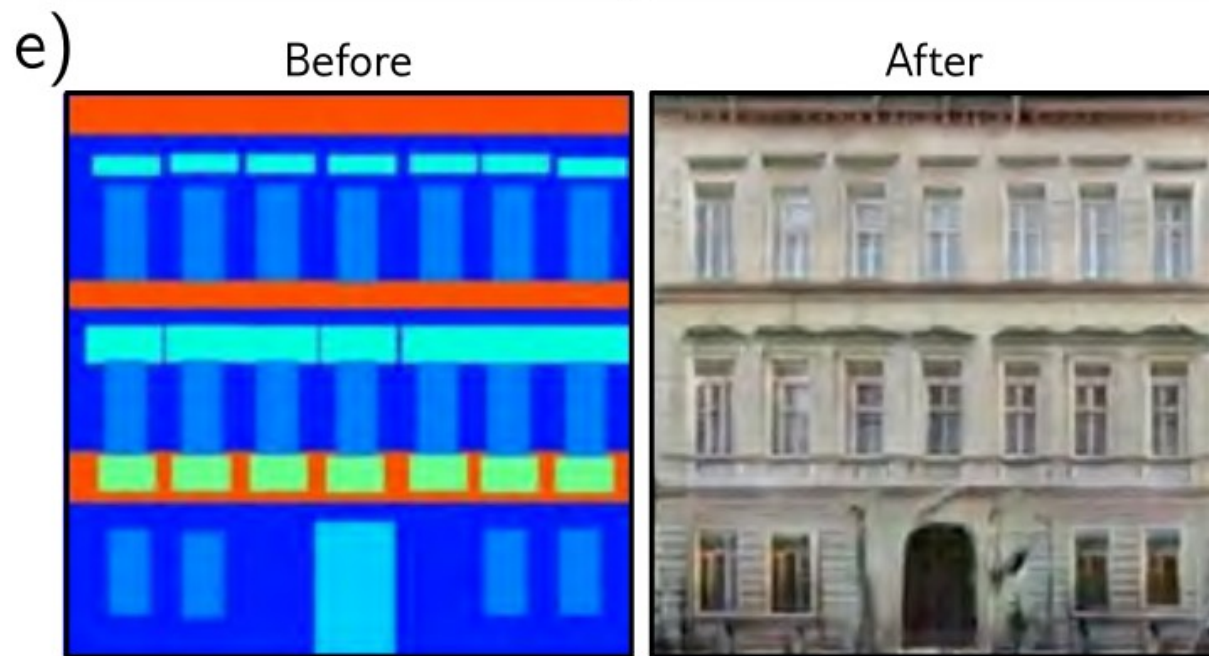
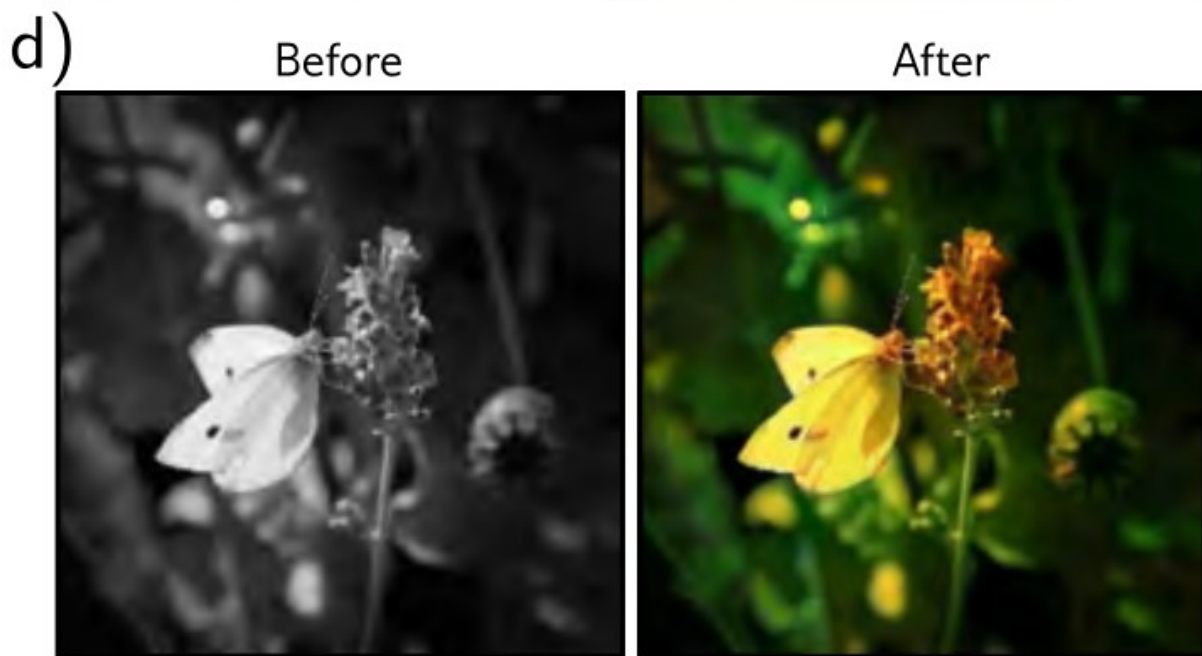
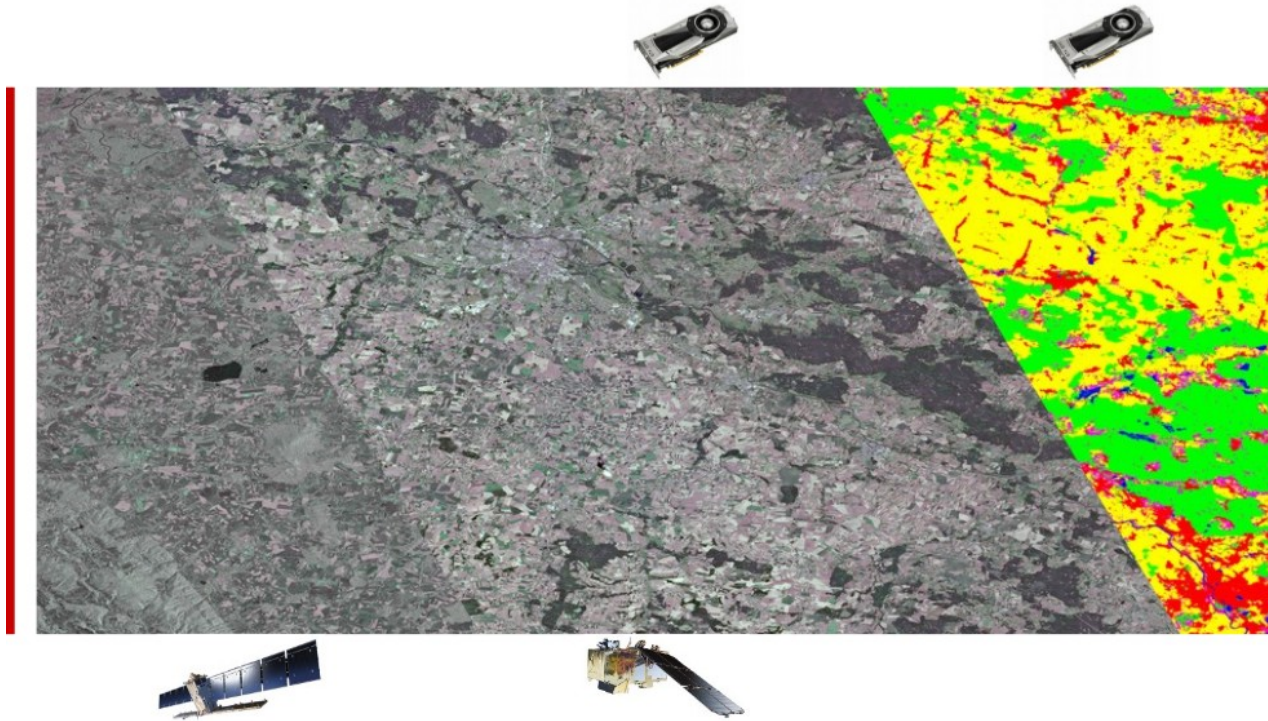


Image Translation: Pix2Pix



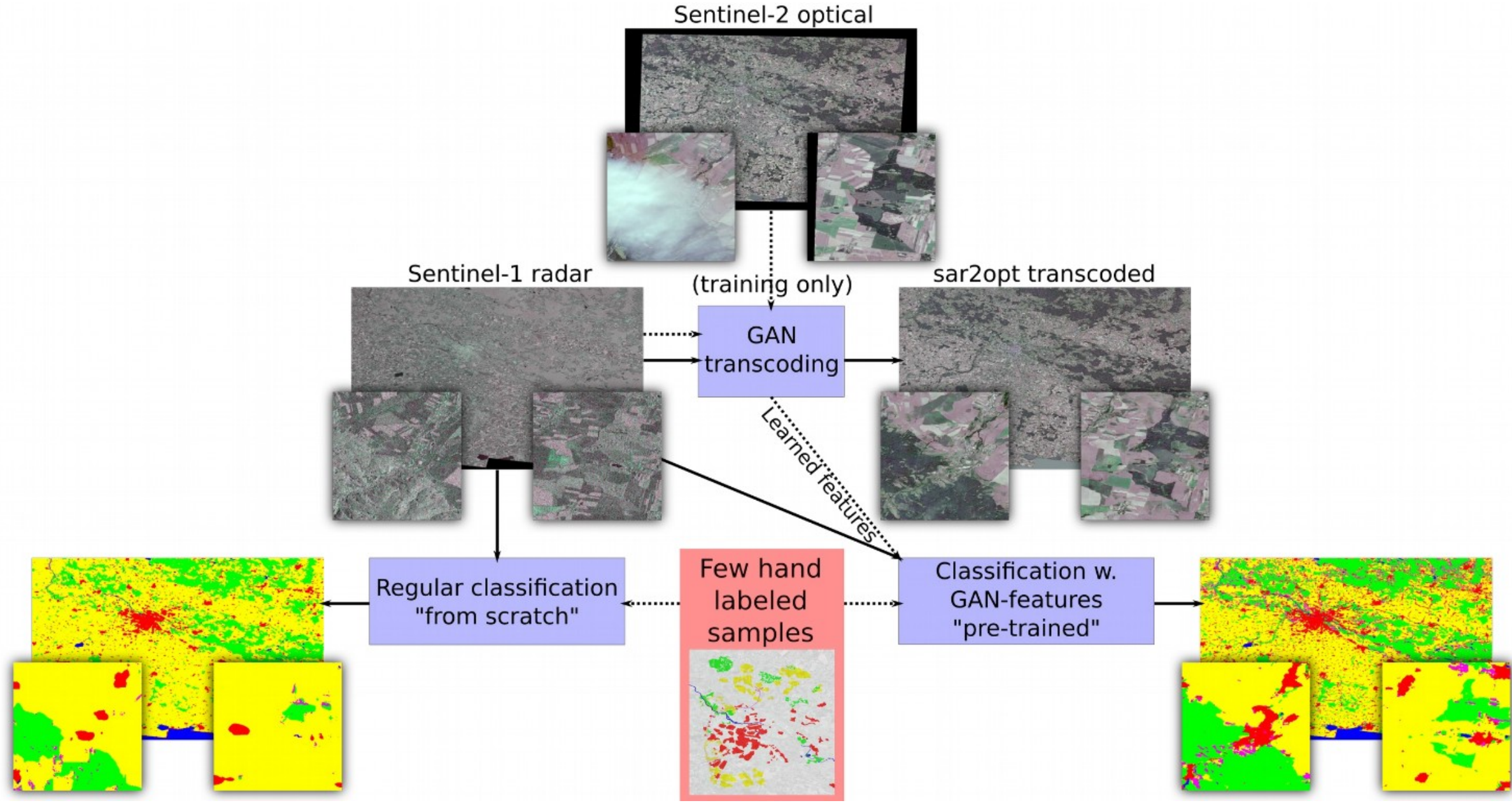


Exploiting GAN-Based SAR to Optical Image Transcoding for Improved Classification via Deep Learning

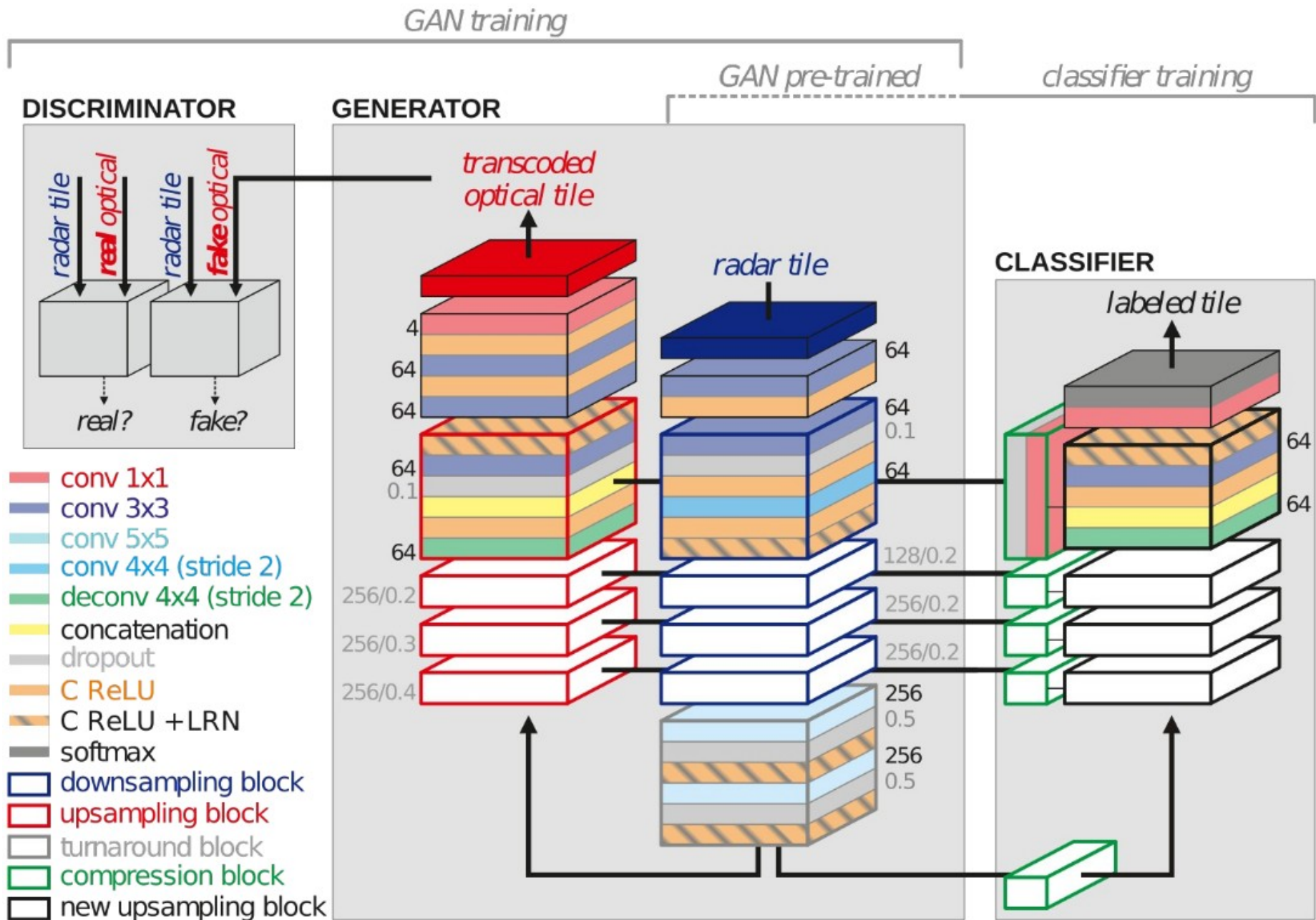
A. Ley, O. D'Hondt, S. Valade, R. Hänsch, O. Hellwich

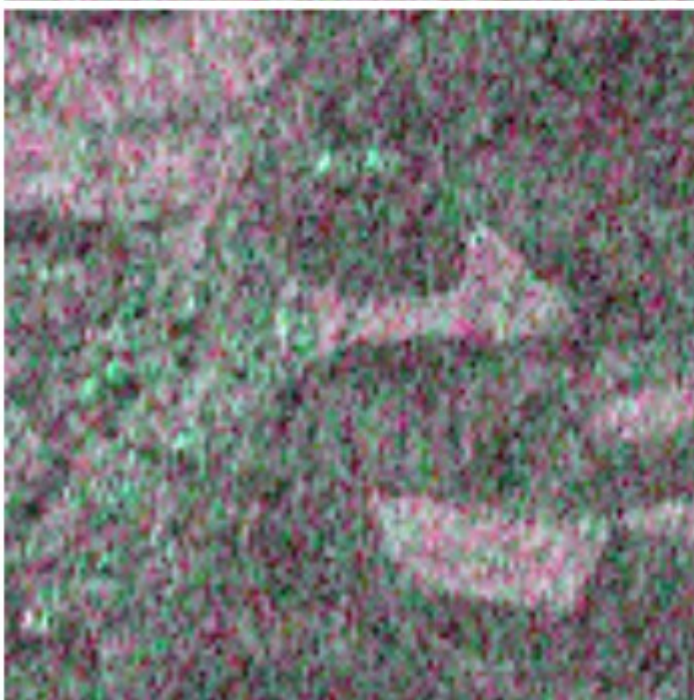
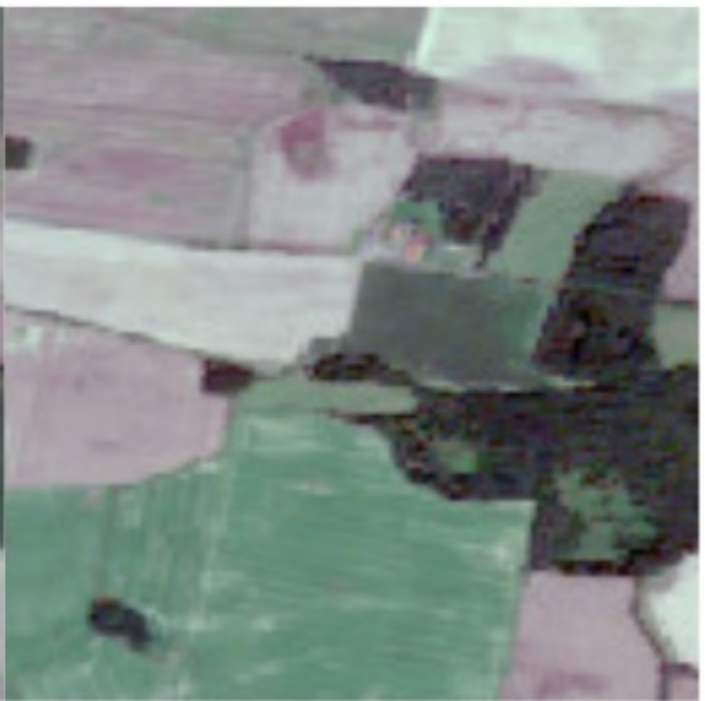
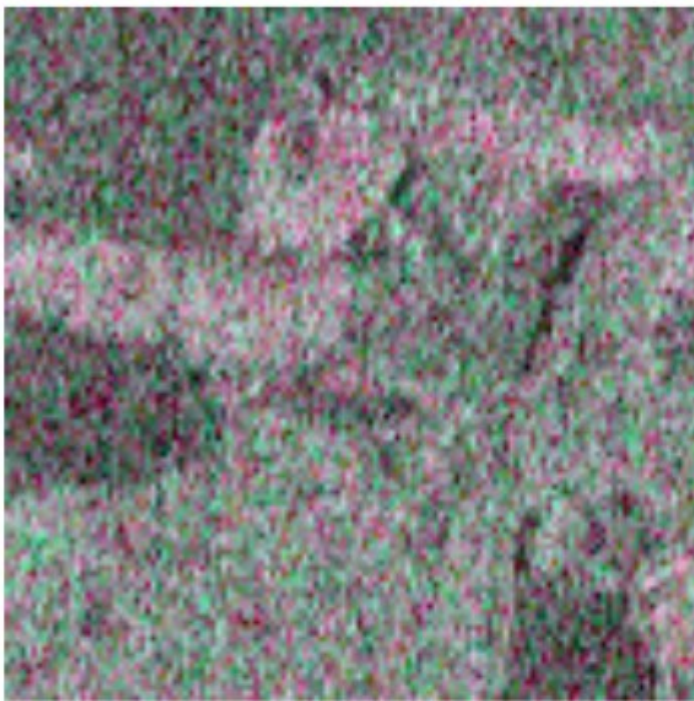
Computer Vision & Remote Sensing | TU Berlin, Germany

Learning from Transcoding



Classification





Classification Results Quantitative

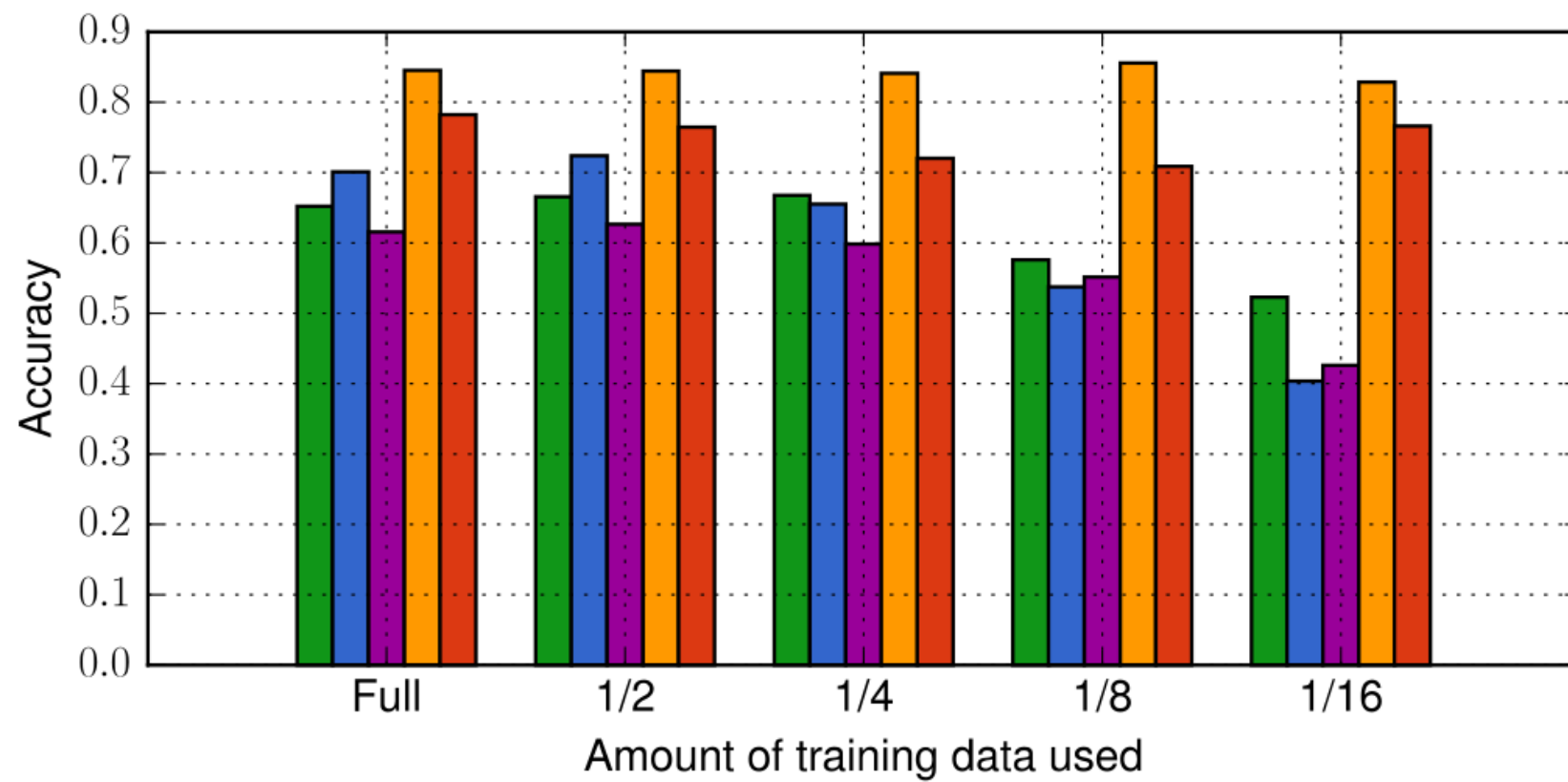
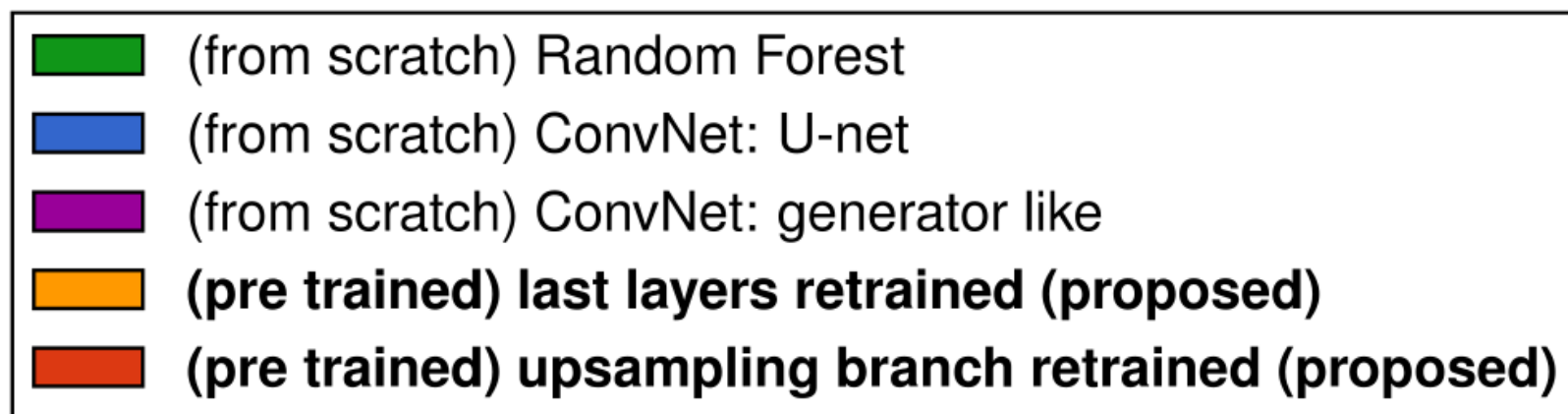


Image Translation: Super-Resolution GAN

- Pix2Pix model attempts to distinguish whether **before/after pair is plausible (real) or not (synthesized)**.
- This requires ground truth to be available.
- **Adversarial loss:** penalty, if a discriminator can distinguish **synthesized image from real image**.

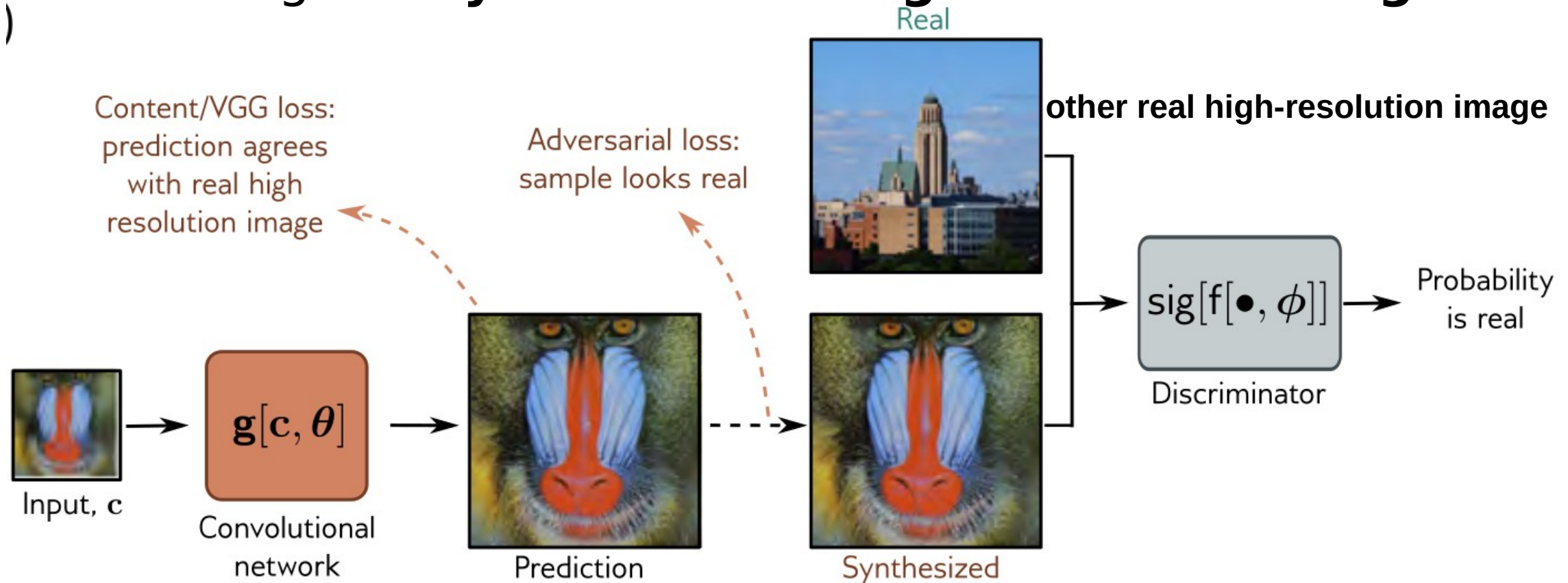


Image Translation: Super-Resolution GAN

- Performance comparison with simple bicubic upsampling

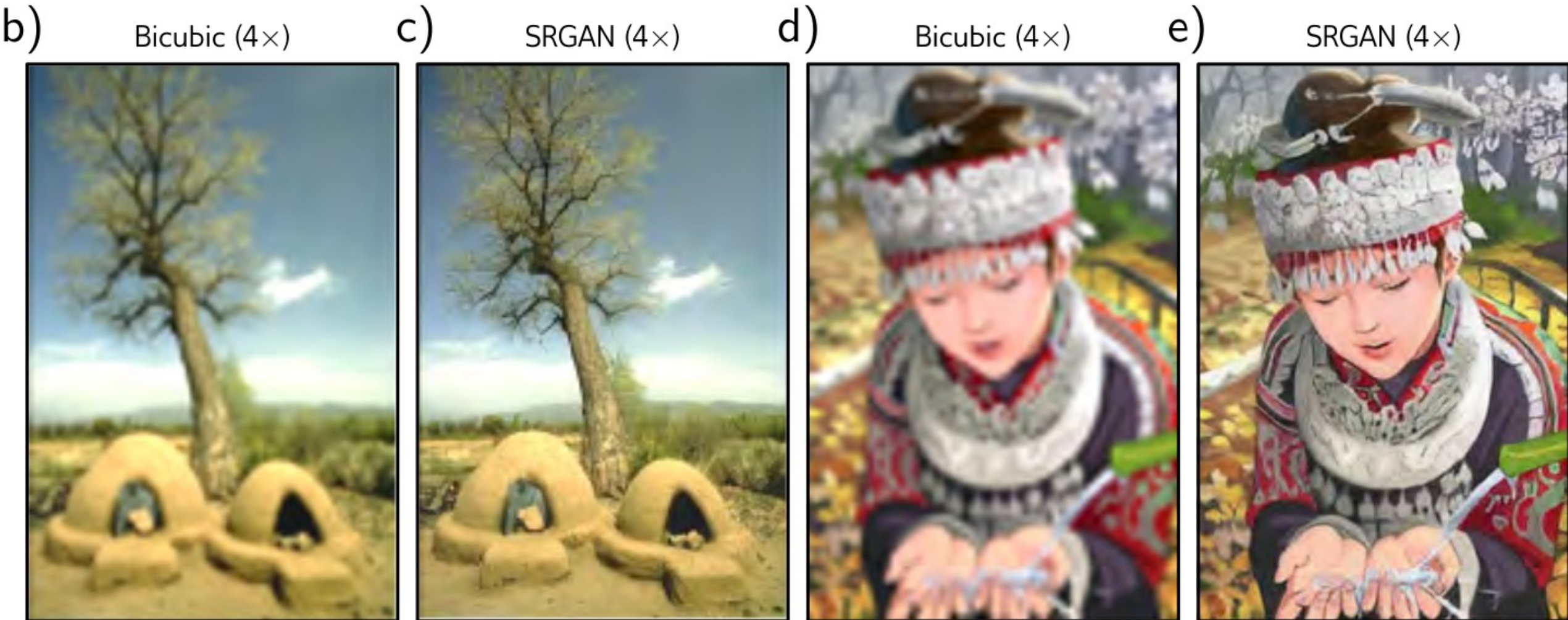


Image Translation: CycleGAN

- We have **two sets of images with distinct styles**, but **no matching pairs**.
- Example: convert a **photography** into an image that looks like a **painting** of Monet's
- Idea: converting **photo to painting**, and then **painting back to photo** should recover the original
- CycleGAN loss is a weighted sum of three losses:
 - **Content loss** encourages before and after image to be similar.
 - **Adversarial loss** encourages after image to be indistinguishable from real examples.
 - **Cycle-consistency** loss encourages mapping to be reversible.

Image Translation: CycleGAN

- CycleGAN loss is a weighted sum of three losses:

a)

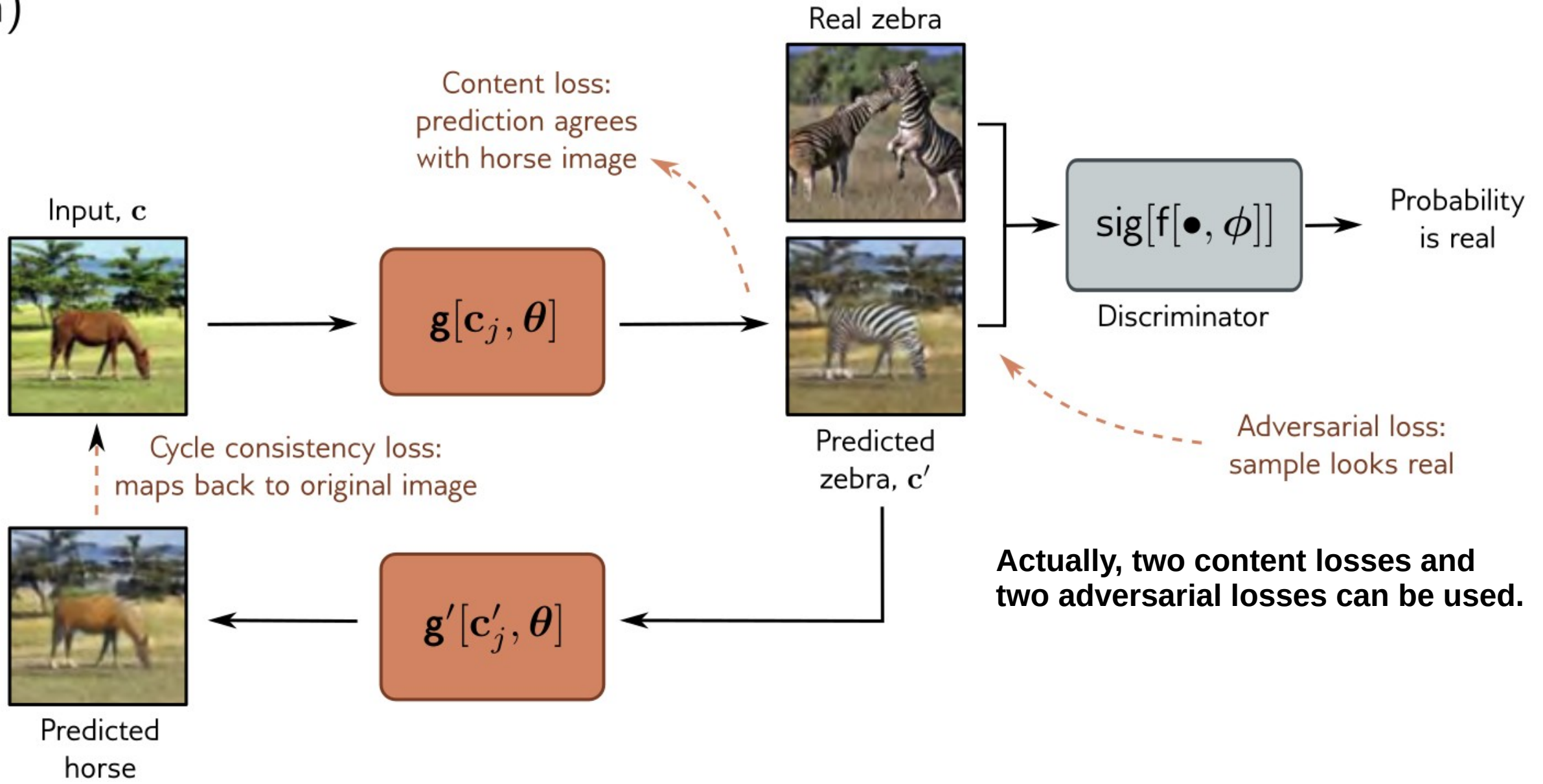


Image Translation: CycleGAN

- CycleGAN loss is a weighted sum of three losses:

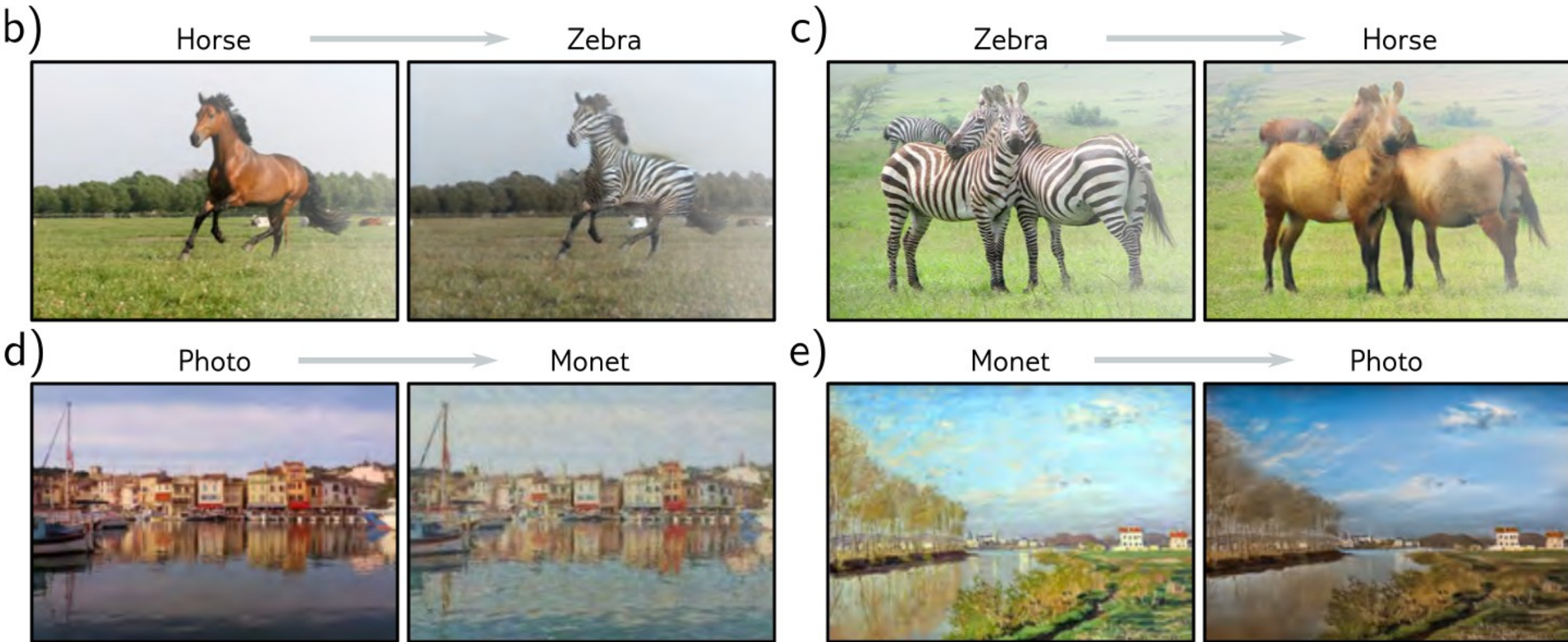


Image Translation: StyleGAN

- StyleGAN controls the output image at different scales and separates style from noise.
- For face images
 - Large-scale changes include face shape and head pose
 - Medium-scale changes include shape and details of facial features
 - Fine-scale changes include hair and skin color.
- **Style** components are **salient to humans**, **noise** represents **unimportant variation**

Image Translation: StyleGAN

- GANs usually start from a latent variable $\mathbf{z}$.
- StyleGANs input latent variables **at various points** of the architecture.

Noise

Main generative pipeline

Style

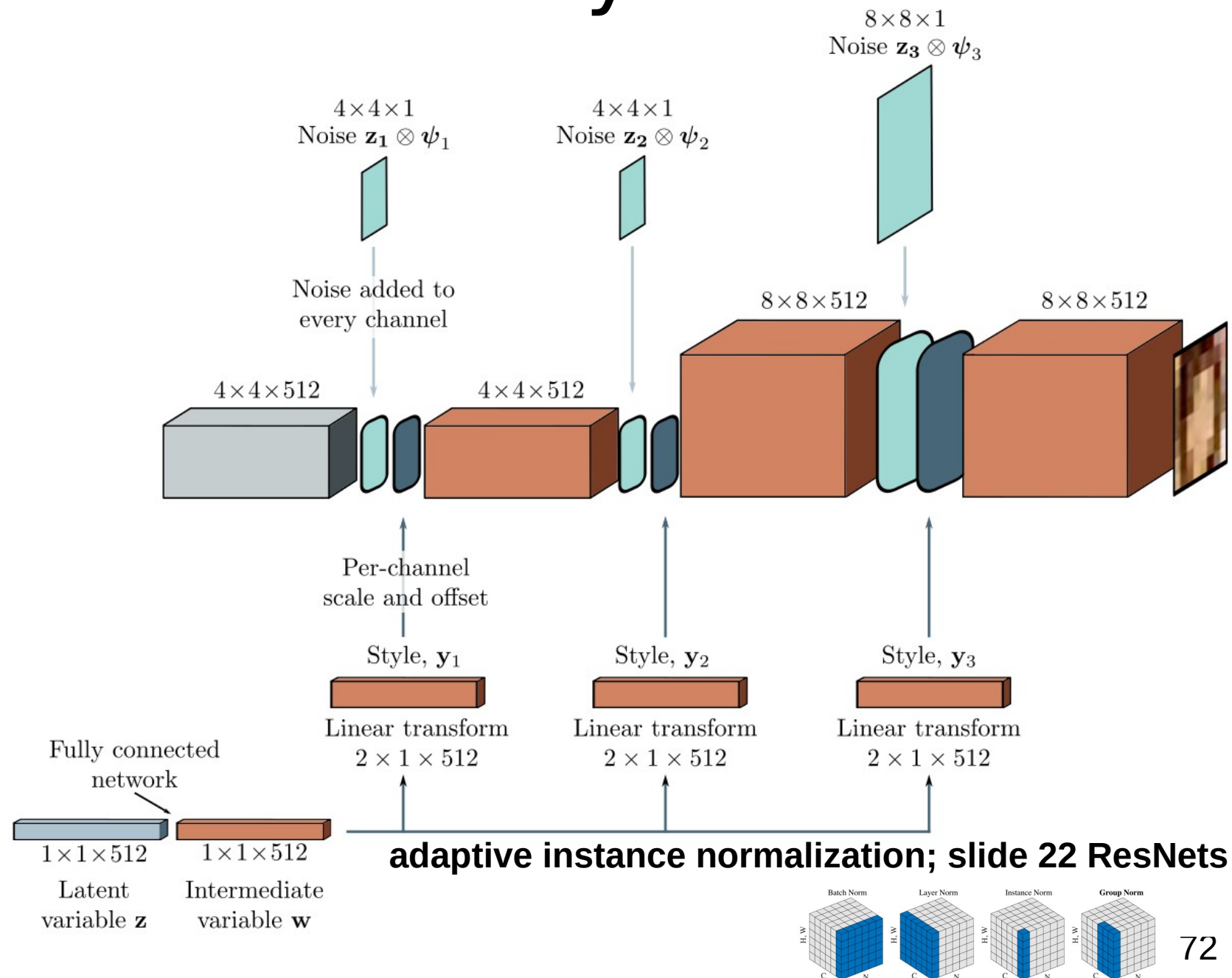


Image Translation: StyleGAN



Changing
all styles

Changing
coarse styles

Changing
med. styles

Changing
fine styles

Increasing
all noise

Changing
coarse noise

Changing
fine noise